DynaGraph: Holistic Optimization for Latency-Critical GraphRAG Serving via Bidirectional Feedback

ABSTRACT

Graph Retrieval-Augmented Generation (GraphRAG) has emerged as a promising paradigm for complex multi-hop reasoning, bridging knowledge graphs with LLMs to answer questions requiring chained evidence. However, deploying GraphRAG in latency-critical serving systems faces a fundamental challenge—the *phase-silo design*: retrievers maximize recall without cost awareness, while serving engines optimize throughput without accounting for query difficulty. This mismatch leads to head-of-line blocking and SLO violations under concurrent and skewed workloads. To address this, we propose *DynaGraph*, a unified framework that jointly optimizes retrieval, aggregation, and scheduling via a closed-loop feedback design. DynaGraph (a) exposes graph-topology features as cost priors, (b) adapts the evidence budget under real-time backpressure, and (c) employs an SLO-aware scheduler to maximize a newly proposed *QA-Goodput* metric that balances answer quality and timeliness; it requires no LLM or retriever finetuning, and operates as lightweight middleware above modern LLM serving engines. Experiments on three multi-hop QA benchmarks across five LLM architectures show that DynaGraph improves QA-Goodput by 2.4×–3.3× and reduces median TTFT by 56%–64% over the strongest GraphRAG baseline, while achieving sub-linear latency scaling under 7× corpus growth. Code, artifacts, and the full version are available at.[1]

1 INTRODUCTION

Online question answering (QA) is rapidly evolving from offline benchmarks to interactive, latency-critical deployments, where systems must answer user queries in real-time using large knowledge sources and language models [31]. Unlike offline evaluation, where latency is largely irrelevant, online QA must deliver responses within strict time budgets while preserving answer quality.

Consider an incident-response copilot used by site reliability engineers during a production outage [6]. An engineer may ask: *"Which customer-facing endpoints are affected by Service X's failure, and what mitigation steps were applied in similar past incidents?"* Answering such queries requires multi-hop reasoning over heterogeneous artifacts: service dependency graphs, runbooks, and incident summaries [13]. Crucially, the system must satisfy a strict *Time-To-First-Token* (TTFT) budget [30, 61], *i.e.,* the latency between issuing a request and receiving the first token of the response. In such high-stakes settings, answers must be not only correct but also delivered within a *Service Level Objective* (SLO) over TTFT (*e.g.,* TTFT ≤ 2 s for 99% of requests) [18]. A late (even correct) answer violates this SLO and breaks the human-in-the-loop cycle where engineers refine diagnostic queries based on prior responses [33, 45].

Example 1: We use an e-commerce customer-support scenario as RE-1: a user reports "payment succeeded but order failed." The copilot must identify affected users, trace the root cause across payment, order, and inventory events, and recommend fixes based on similar past incidents. Answering this query requires multi-hop reasoning over heterogeneous evidence sources (*e.g.,* transaction records, order-service failure traces, and historical tickets and runbooks), by traversing and aggregating distributed evidence under strict latency constraints. The answer is useful only if it is both correct and delivered within the TTFT/SLO budget. □

To handle such complex queries, prior work has evolved along two directions, as summarized in Figure 1. On the *retrieval side*, GraphRAG paradigms [16, 34] and neuro-symbolic approaches (*e.g.,* HippoRAG [19]) capture long dependency chains by constructing dense subgraphs or leveraging Personalized PageRank. These methods improve reasoning depth and reduce hallucinations by incorporating richer structural context, but primarily optimize retrieval recall (*i.e.,* the fraction of relevant evidence retrieved) with limited awareness of downstream execution cost. Thus, they often retrieve large contexts (*e.g.,* dense neighborhoods near "supernodes"), increasing prefill latency and inference memory footprint.

Conversely, on the *serving side*, systems such as vLLM [30] and SGLang [60] improve inference throughput via techniques like PagedAttention and RadixAttention, enabling efficient continuous batching and prefix sharing. Recent systems (*e.g.,* RAGO [26] and RAGCache [27]) further optimize configuration and caching for RAG workloads. However, these approaches linearize retrieved evidence into a flat token sequence, ignoring graph topology and per-query reasoning difficulty. As a result, they incur near-uniform compute even when only a small subset of evidence suffices.

Although emerging efforts have attempted to bridge this gap, they remain constrained by unidirectional data flows. For instance, DepCache [54] leverages dependency graphs to prune attention computation during inference, and FusionRAG [47] optimizes context splicing for cache reuse. Yet, these methods operate reactively at the serving layer; they cannot proactively regulate the upstream retrieval budget based on real-time system contention, nor can they prioritize critical reasoning paths under strict latency constraints.

This disconnect leads to a brittle *"Phase-Silo" Design* that optimizes intermediate metrics misaligned with end-user experience:

- Retrieval blindness: Retrieval modules maximize recall without accounting for serving-time cost. Expanding dense neighborhoods can inflate context length, triggering retrieval floods that immediately violate TTFT SLOs.
- Serving agnosticism: Schedulers optimize *throughput* (tokens generated per second) but ignore *goodput*, *i.e.,* the sustainable request rate under latency SLOs [61]. This can prioritize short queries while starving multi-hop ones, causing timeouts or incomplete reasoning.

This problem has received sustained attention in the data management community [1, 42, 47, 54]. The core challenge is cross-layer coordination: *how much evidence to retrieve per query and when to schedule it, guided by data structure and system state.*

Our proposal. To bridge these silos, we propose DynaGraph, a

[1] https://anonymous.4open.science/r/DynaGraph-Releasee-0496.

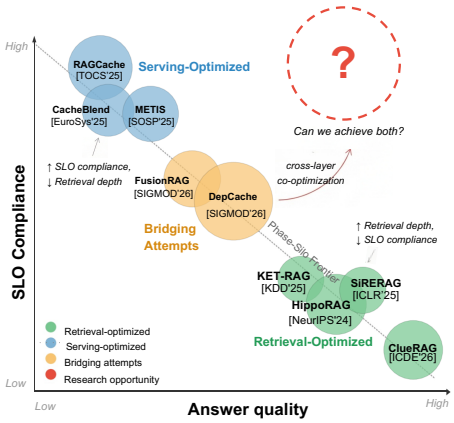


Figure 1: Quality–latency trade-off in RAG. Retrieval-optimized methods [16, 19, 22, 57] (green) boost answer quality but often miss SLOs; serving-optimized methods [26, 27, 40, 53] (blue) reduce latency but weaken retrieval depth. Bridging efforts [47, 54] (orange) narrow the gap but lack cross-layer feedback. Can we get both?

holistic serving framework that turns linear RAG into a cooperative closed loop. Rather than optimizing retrieval recall and serving throughput in isolation, DynaGraph establishes a *bidirectional feedback channel* between the data and compute layers, using graph topology as a serving-time control signal to jointly coordinate evidence budgeting and scheduling under SLOs, where the topology reflects retrieval difficulty and predicts cost sensitivity, yielding a unified control prior for both decisions.

DynaGraph *redesigns retrieval to be compute-aware.* Instead of forwarding raw chunks, the retriever characterizes each retrieved subgraph's topological complexity (*e.g.,* high-degree "supernodes" or dense cliques) and exposes these signals as *cost priors* for scheduling. We also use a lightweight cost model that maps per-query evidence budgets to predicted prefill latency, enabling the aggregation controller and scheduler to estimate TTFT impact before inference.

DynaGraph *introduces serving-driven backpressure.* The inference engine monitors SLO slack and hardware contention and, as saturation approaches, feeds back to the aggregation layer to tighten evidence budgets for new queries. This turns a rigid recall–latency conflict into an adaptive trade-off: prune context for latency-sensitive workloads while reserving compute for hard multi-hop queries that require deeper evidence chains.

In summary, this paper makes the following contributions:

- *Problem formulation with a new metric (Section 2):* We introduce *QA-Goodput*, a quality-weighted serving metric that couples answer quality with hard TTFT deadlines. Based on this metric, we formulate a joint optimization problem that *simultaneously* selects per-query evidence budgets and an SLO-aware serving order to maximize QA-Goodput, and we show that this problem is NP-hard.
- *Preliminary study (Section 3):* Through a pilot online serving study, we show that phase-silo GraphRAG breaks down under strict and heterogeneous SLOs: fixed retrieval/aggregation budgets cannot handle skewed per-query cost and difficulty; scheduling alone mainly redistributes latency without reducing load; and congestion leads to head-of-line blocking that degrades end-to-end availability. These findings motivate a *joint*, feedback-driven design that couples cost control with SLO-aware scheduling.
- *Closed-loop pipeline (Section 4):* We design DynaGraph, a bidirectional feedback pipeline with three phases: (1) In Phase 1 (*Sensor*), it extracts lightweight topological signals during retrieval which can be used as serving-time control signals for downstream budgeting and scheduling; (2) In Phase 2 (*Controller*), it performs backpressure-driven adaptive aggregation, leveraging a lightweight cost model to estimate request cost and determine per-query budgets under real-time congestion guided by topology-derived signals; (3) In Phase 3 (*Actuator*), it applies an SLO-aware scheduler that ranks queries by urgency and estimated prefill cost with topology-informed difficulty, while feeding congestion signals back to the controller.
- *Extensive evaluation (Section 5):* We implement DynaGraph as a middleware that requires no LLM or retriever fine-tuning at serving time and that is portable across models and serving backends, requiring only lightweight calibration atop modern LLM serving engines, and evaluate it across benchmarks, models, workloads, and SLO regimes. Across three benchmarks and five LLM architectures, DynaGraph achieves 2.4×–3.3× higher QA-Goodput and reduces median TTFT by 56%–64% over the strongest baseline, while maintaining sub-linear end-to-end latency under 7× corpus growth.

We review related work in Section 6 and conclude in Section 7.

2 PROBLEM FORMULATION

In this section, we first introduce the standard graph-enhanced RAG pipeline, then present a new metric, *QA-Goodput*, that couples answer quality with hard TTFT deadlines, and finally formalize the problem studied in this paper.

2.1 Graph-Enhanced RAG Pipeline

Knowledge corpus. The knowledge corpus can be modeled as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$, where $\mathcal{V}$ is the set of nodes representing granular knowledge units (*e.g.,* documents, chunks, and entities), $\mathcal{T}_v$ specifies node types (*e.g., entity, document, chunk*), $\mathcal{E}$ is the set of edges capturing pairwise relationships between nodes, and $\mathcal{T}_e$ defines edge types (*e.g.,* hyperlink, co-occurrence, semantic similarity) [16].

Pipeline. Given a graph-modeled corpus, a graph-enhanced RAG pipeline answers a multi-hop query q in three phases:

Phase 1: Retrieval (R). The retriever constructs a query-specific subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ by combining embedding similarity with graph traversal (*e.g.,* community detection [16]). $\mathcal{G}_q$ captures multi-hop evidence paths and exposes topological signals $\mathcal{F}_q$ (*e.g.,* connectivity density and bridge nodes) reflecting reasoning complexity, which prior work typically discards [16, 19, 34].

Phase 2: Aggregation (A). Given retrieved $\mathcal{G}_q$, the aggregator selects K evidence chunks X_{ctx} to form the prompt $P = [X_{\text{sys}}, X_{\text{ctx}}, q]$, where X_{sys} provides task-specific instructions. Most systems fix K, inducing a rigid trade-off between context richness and cost. Let L_q denote the prompt length, which determines prefill cost and TTFT. Since K dominates, we approximate $L_q \approx |X_{\text{sys}}| + K \cdot \bar{\ell} + |q|$, where $\bar{\ell}$ is the average chunk length in X_{ctx}.

Phase 3: Generation (G) and Serving. Given prompt P, the LLM generates answer Y via prefill (KV cache construction, which domi-

nates TTFT) followed by decoding. Prefill cost grows super-linearly with L_q, so larger K increases latency. In practice, queries arrive stochastically at rate $\lambda(t)$ with heterogeneous complexity (*e.g.,* K=5 for a single-hop lookup vs. K=20 for a multi-hop query), inducing large cost variance. Under concurrency, requests compete for GPU resources, requiring batching, KV caching, and scheduling to meet latency targets. We focus on locally served open-source LLMs (*e.g.,* Qwen, Mistral) to enable fine-grained control.

Example 2: Continuing Example 1, the query is handled by a standard three-phase pipeline:
∘ *Phase 1 (Retrieval):* Starting from payment and order entities, the retriever extracts a subgraph $\mathcal{G}_{q'}$ linking transactions, service dependencies, failure traces, and incident signatures. The presence of a high-degree node (*e.g.,* "Error 503") indicates a dense fault region requiring more evidence to resolve.
∘ *Phase 2 (Aggregation):* The aggregator selects K chunks (*e.g.,* = 5) from $\mathcal{G}_{q'}$ to form $P_{q'} = [X_{\text{sys}}, X_{\text{ctx}}, q']$, where small K may miss cross-service dependencies; and large K increases cost and latency.
∘ *Phase 3 (Generation and Serving):* The engine executes $P_{q'}$ via prefill and decoding. Under FCFS scheduling, this query may be delayed by low-urgency requests, risking TTFT/SLO violations. □

Limitations. Modern serving engines (*e.g.,* vLLM [30] and SGLang [60]) improve throughput via KV-cache management, but remain inadequate for GraphRAG workloads with highly variable query difficulty and context size (see Section 3).

Cost-Blind Scheduling. Schedulers (*e.g.,* FCFS) treat requests as homogeneous, ignoring cost variance from topology, prompt length, and reasoning depth. For instance, a multi-hop query over a dense subgraph (*e.g.,* a supernode) may require far more context and prefill time than a single-hop lookup, yet both receive equal priority. This leads to *head-of-line (HOL) blocking* [28], where expensive queries delay latency-sensitive ones (or vice versa).

Static Context Budget. Most pipelines fix the context budget K, selecting the same number of chunks regardless of query difficulty or system load. Large K inflates prefill latency under contention, while small K wastes capacity and harms quality under light load. No existing system adapts K to real-time conditions.

No Cross-Layer Feedback. Retrieval, aggregation, and serving operate as an open loop: the serving layer cannot feed congestion signals upstream to adjust K, and upstream stages cannot provide cost signals to guide scheduling. As a result, the system cannot adapt to load fluctuations or heterogeneous query complexity.

Failure modes. Given the above limitations, existing approaches exhibit two common failure modes.

Fast but wrong (mode (a)). With insufficient context (*i.e.,* small K), LLM lacks key evidence and may hallucinate [3]. For reasoning-augmented models, missing grounding can also trigger longer exploration, increasing decode cost without improving accuracy [32].

Accurate but late (mode (b)). With excessive context (*i.e.,* large K), prefill dominates latency: answers are often correct but arrive late.

Both modes render responses ineffective: (a) arises for complex queries with small K, while (b) occurs for simple ones with large K.

Example 3: Continuing Example 2 with an SLO threshold τ=2 s, consider query under two static configurations:
∘ With K=5, the retriever retains only payment-callback logs and misses the upstream inventory-lock root cause; the LLM thus hallucinates a "gateway timeout" (TTFT ∼180 ms; failure mode (a)).
∘ With K=20, all relevant evidence is included and the root cause is correctly identified, but the ∼4,000-token prompt raises TTFT to ∼2,800 ms, exceeding the SLO (failure mode (b)). □

2.2 Metric and Problem Formulation

In this subsection, we introduce a new evaluation metric for graph-enhanced RAG, upon which we formalize our problem.

Metric. We first review the widely used Goodput metric and its limitations, and then introduce a new metric for graph-enhanced RAG that jointly accounts for answer accuracy and a hard deadline.

Goodput [61]. It quantifies the maximum sustainable request arrival rate subject to a latency Service Level Objective (SLO), specified as a per-request deadline τ_q on TTFT (time-to-first-token):

$$\text{Goodput} = \max\ \lambda \quad \text{s.t.} \quad \Pr\big(\text{TTFT} \le \tau_q\big) \ge \beta \tag{1}$$

where λ is the request arrival rate and $\beta \in (0, 1]$ is the target SLO-attainment probability.

As shown in Eq. (1), Goodput accounts only for timeliness; it treats a correct on-time answer and an on-time hallucination equally, leaving failure mode (a) unaddressed.

QA-Goodput. To capture the failure modes in Section 2.1, we introduce *QA-Goodput* $\mathcal{J}_\lambda$, which (a) assigns zero credit to any response that misses its deadline, regardless of correctness, and (b) penalizes low-quality answers. Formally, for a query set $\mathcal{S} = \{q_i\}_{i=1}^N$ at arrival rate λ, where each query q_i has a TTFT deadline τ_{q_i}, we define:

$$\mathcal{J}_\lambda = \frac{1}{N} \sum_{i=1}^{N} \underbrace{\text{Score}(q_i)}_{\text{quality}} \cdot \underbrace{\mathbb{I}(\text{TTFT}_i \le \tau_{q_i})}_{\text{timeliness}} \tag{2}$$

where $\text{Score}(q_i) \in [0, 1]$ measures answer quality (*e.g.,* token-level F1 [39]), and $\mathbb{I}(\cdot)$ is an indicator function. Intuitively, a higher $\mathcal{J}_\lambda$ indicates that more queries in $\mathcal{S}$ are answered accurately and within their deadlines, thereby mitigating both failure modes.

Note. (1) Goodput considers only timeliness, counting any on-time response as successful regardless of correctness. In contrast, QA-Goodput couples quality with a hard deadline, assigning zero credit to late responses and penalizing low-quality ones. (2) QA-Goodput captures settings requiring *both* high answer quality and strict deadline compliance, complementing Goodput.

Problem. We formalize the problem studied in this paper as follows.
∘ *Input:* a document corpus $\mathcal{D}$ (from which a heterogeneous graph $\mathcal{G}$ is constructed, and a query batch $\mathcal{S} = \{q_i\}_{i=1}^n$ arriving at rate λ, where each query q_i has a TTFT deadline τ_{q_i} and an admissible retrieval/aggregation budget $K_i \in [K_{\min}, K_{\max}]$.
∘ *Output:* a serving order π of queries in $\mathcal{S}$ and per-query budgets $\mathcal{K} = \{K_1, \ldots, K_n\}$.
∘ *Objective:* maximize QA-Goodput under query budget constraints:

$$\max_{\mathbf{K},\,\pi} \ \sum_{i=1}^{n} Q(K_i, q_i) \cdot \mathbb{I}(\text{TTFT}_i \le \tau_{q_i}) \quad \text{s.t.} \ \ K_i \in [K_{\min}, K_{\max}],\ \forall i \tag{3}$$

where $Q(K_i, q_i)$ denotes the answer score (cf. Eq. (2)) when serving

q_i with K_i evidence chunks.

Note. (1) The problem requires jointly optimizing two coupled decisions: the serving order π (scheduling) and the per-query budgets $\mathcal{K}$ (budget allocation). (2) The interval $[K_{\min}, K_{\max}]$ defines feasible system budgets and can be set from prior workload knowledge. (3) Since scheduling and budget allocation jointly influence both quality and timeliness, the problem is more challenging than standard deadline scheduling with fixed processing times (see below).

Hardness. The restricted decision version of the studied problem in Eq. (3) is to decide, given a query set $\mathcal{S}$, arrival rate λ, per-query deadlines τ_{q_i} for each $q_i \in \mathcal{S}$, a fixed budget vector $\mathbf{K}$, and a threshold η, whether there exists a serving order π on a non-preemptive single machine such that $\sum_{i=1}^{n} Q(K_i, q_i) \cdot \mathbb{I}(\mathsf{TTFT}_i \leq \tau_{q_i}) \geq \eta$. We show that even this restricted version is intractable.

Theorem 1: *The above restricted decision problem is NP-hard.* □

Proof. We establish NP-hardness by reducing from Partition [29]. Proof details are deferred to Appendix B.1 in [4].

3 MOTIVATION AND ANALYSIS

In this section, we present the test setup (Section 3.1), empirical evidence of their limitations (Section 3.2), and key observations motivating our approach (Section 3.3).

3.1 Setup

Dataset. We use 2WikiMultihopQA [21], a two-hop QA benchmark with four reasoning types: *compositional* (n=413), *comparison* (n=244), *bridge_comparison* (n=235, requiring cross-document bridging and comparison), and *inference* (n=108). We treat *bridge_comparison* as "Hard" and the remaining types as "Easy", as they are less sensitive to the context budget K. This dataset's annotated reasoning types enable controlled stratification by difficulty.

Workload. Requests arrive via a Poisson process at rate λ [27, 40], with queries drawn from a Zipfian popularity distribution (α=1.0) to reflect the skewed entity distribution observed in knowledge-graph workloads [7, 8]. This amplifies prompt-length variance and HOL blocking, leading to a harder scheduling problem than uniform.

Configurations. We evaluate four settings: *Static (C1)* uses fixed K=20 with FCFS (phase-silo baseline); *Schedule-Only (C2)* replaces FCFS with SLO-aware scheduling, isolating scheduling; *Adaptive-K (C3)* applies difficulty-aware K with FCFS (expert-provided difficulty), isolating budget adaptation; *Joint (C4)* combines adaptive K, SLO-aware scheduling, and load-dependent backpressure under expert-defined policies. These isolate *budget adaptation* (C1→C3) and *scheduling* (C1→C2), with C4 evaluating their joint effect.

Infrastructure. All experiments run on a single NVIDIA RTX 4090 (24 GB). We use Qwen3-4B with Zipfian arrivals (λ=30 req/s; 1000 queries × 3000 requests; concurrency 64).

Quality Metric. Answer quality is measured by token-level F1 [39], corresponding to Score(q_i) in the QA-Goodput definition (Eq. 2).

3.2 Observations

Under the test setup in Section 3.1, we observe the following.

(1) The context size K matters. We varied the aggregation budget

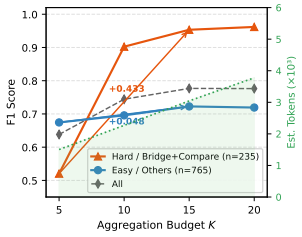


Figure 2: Static configuration under varying K. Green shading: prefill tokens (right axis).

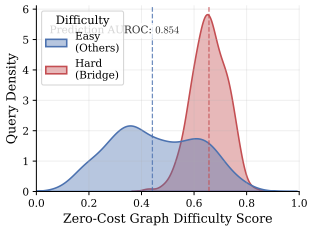


Figure 3: Zero-cost difficulty prediction. Difficulty score distribution for Easy (n=765) and Hard (n=235); AUROC = 0.854.

K (5–20) for the static configuration (C1) across four reasoning types (Figure 2). (a) The quality–cost trade-off is strongly difficulty-dependent: hard queries gain substantially from larger K, while easy queries gain little. For instance, increasing K from 5 to 15 raises hard-query F_1 from 0.52 to 0.95, *e.g.*, 9× the gain for easy queries, since small K often misses evidence for multi-step reasoning. (b) Larger K can also hurt: increasing K from 15 to 20 reduces F_1 for easy queries, suggesting that oversized contexts introduce noise. Thus, fixed-K aggregation wastes budget on easy queries and underserves hard ones, making difficulty-aware K essential.

(2) Adaptive K alone improves efficiency but may violate strict SLOs. We compared adaptive-K (C3) against Static (C1) to test whether difficulty-based K improves efficiency without harming accuracy or SLOs. In Table 1, with expert difficulty labels, C3 reduces avg K from 20.0 to 14.7 (−26%) with negligible F1 loss (−0.008). Under a moderate SLO (τ=500 ms), C3 reaches 91.9% compliance and $\mathcal{J}_\lambda$@500 = 74.0% (+26.8pp over C1); under a strict SLO (τ=200 ms), it still misses 54% of deadlines ($\mathcal{J}_\lambda$@200 = 38.3%). This is because FCFS ignores urgency: adaptive K reduces per-query cost but cannot prevent HOL blocking. Moreover, C3 assumes serving-time difficulty labels, which is unrealistic; prior methods [43] rely on similarity signals and lack load-aware feedback.

(3) Scheduling alone is insufficient. Since FCFS still violates strict SLOs even with adaptive K (Observation (2)), we evaluated whether scheduling alone can close the gap. Configuration C2 applies an SLO-aware priority policy while keeping K=20 fixed. In Table 1, prioritization reduces median latency (P50: 426→361 ms) but worsens the tail (P99: 2,796→6,484 ms, 2.3×), because deprioritized queries still pay the full prefill cost and can wait unbounded under contention. As a result, QA-Goodput improves only slightly: $\mathcal{J}_\lambda$@200 increases by +2.5pp (16.3→18.8%), far below the +22.0pp gain from C3. Thus, scheduling reshapes latency but cannot reduce prefill cost.

(4) Joint optimization is promising. Combining adaptive K and SLO-aware scheduling (C4) yields the best results: $\mathcal{J}_\lambda$@200 = 61.4% (3.8× over C1), $\mathcal{J}_\lambda$@500 = 78.4%, and lowest P50 (136 ms, −68%), with small F1 loss (−0.017). Improvements over C3 (+28.5pp SLO@200; +4.4pp $\mathcal{J}_\lambda$@500) show that scheduling amplifies adaptive K. However, C4 depends on hand-tuned labels/policies, limiting practicality.

(5) Query difficulty can be captured by graph topology. To make C4 practical, we need to estimate per-query difficulty at serving time without expert labels or extra inference. Prior work shows graph properties correlate with QA difficulty [24, 52, 56]. We therefore use retrieval-time subgraph statistics—zero-cost byproducts of graph

Table 1: Motivation: configuration comparison. (Qwen3-4B; 2WikiMultiHopQA; Zipf λ=30 req/s; 1000q×3000 requests.)

Config	Quality		Latency (↓)		τ=200 ms (↑)		τ=500 ms (↑)	
	F1	avg K	P50	P99	SLO	$\mathcal{J}_\lambda$	SLO	$\mathcal{J}_\lambda$
Static (C1)	0.758	20.0	426 ms	2796 ms	18.7%	16.3%	57.2%	47.2%
Sched-Only (C2)	**0.762**	20.0	361 ms	6484 ms	21.6%	18.8%	66.0%	54.9%
Adaptive-K (C3)	0.750	14.7	218 ms	**2224 ms**	45.9%	38.3%	91.9%	74.0%
Joint (C4)	0.741	**12.5**	**136 ms**	2288 ms	**74.4%**	**61.4%**	**97.8%**	**78.4%**

traversal—as proxies. As shown in Figure 3, a linear combination of three topological features (Section 4.2) achieves AUROC = 0.854 in separating Hard from Easy queries, without LLM inference or task-specific training. This is because multi-hop queries induce structurally distinct subgraphs, largely independent of surface phrasing.

3.3 Summary and Insights

Summary. The key findings from Section 3.2 are as follows.

(1) *Adaptive aggregation is the primary lever for the quality–cost trade-off.* Observation (1) shows that a fixed context budget wastes tokens on easy queries while under-serving hard ones. Difficulty-aware budgeting reduces avg K from 20 to 14.7 with only −0.008 F1 loss, yielding 2.4× higher $\mathcal{J}_\lambda$@200ms than Static, and significantly improving latency (P50: 426→218 ms, −49%).

(2) *Strict SLOs require scheduling coupled with backpressure.* Observations (2–3) show that scheduling alone improves median latency but worsens the tail (P99: 6,484 ms). In contrast, combining SLO-aware scheduling with adaptive K and backpressure feedback yields an observable gain in SLO@200ms over Adaptive-K alone (45.9%→74.4%), as backpressure compresses budgets under congestion and prevents queue buildup.

(3) *Graph topology provides accurate, zero-cost difficulty signals.* Observation (5) shows that retrieval-time subgraph features separate Hard vs Easy with AUROC = 0.854 (Obs. 5), enabling difficulty scoring and priority assignment without extra model inference.

Insights. (a) Graph topology effectively captures query difficulty, enabling lightweight, retrieval-time signals to guide budgeting and replace impractical expert tuning; we thus need to study which features are most predictive and stable. (b) Beyond controlled settings, real deployments involve large, heterogeneous workloads with varying deadlines, requiring joint, online adaptation of budgets and scheduling; achieving high QA-Goodput thus necessitates a closed-loop coupling of aggregation and serving.

4 METHODOLOGY

In this section, we overview our system (Section 4.1), detail its main components (Sections 4.2–4.4), and present a closed-loop algorithm with guarantees (Section 4.5).

4.1 System Overview

Overview. We present DynaGraph, a holistic serving framework for maximizing QA-Goodput (Section 2.2) under stochastic arrival loads. As shown in Figure 4, unlike feed-forward pipelines, it establishes a *bidirectional feedback loop* between data complexity (Graph) and system capacity (Serving), and decomposes the joint problem (Eq. 3) into three coordinated phases.

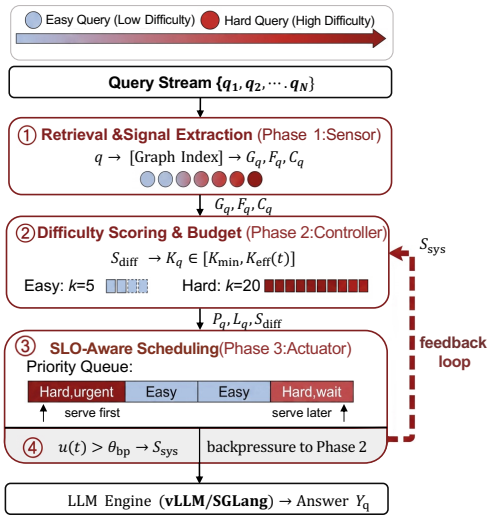


Figure 4: DynaGraph Architecture. Phase 1 (Sensor) extracts topological priors from the graph index; Phase 2 (Controller) budgets context K_q based on difficulty and backpressure; Phase 3 (Actuator) schedules by urgency and cost, feeding congestion signals back to Phase 2.

Given a query q, the system first retrieves a query-specific subgraph and extracts lightweight topological features that capture its structural complexity (Phase 1). These signals are then used to estimate query difficulty and determine a budgeted context and prompt under current system load (Phase 2). Finally, the resulting requests are scheduled based on their urgency, estimated cost, and topology-informed difficulty, producing the execution order and inference results (Phase 3). This forms a closed loop in which topology-derived signals guide both budgeting and scheduling decisions.

The key ideas of the three phases are as follows.

Phase 1: Retrieval & Signal Extraction (Sensor, Section *4.2).* Given query q and corpus $\mathcal{D}$, this phase produces: (a) a heterogeneous graph $\mathcal{G}$ (offline), (b) a query-induced subgraph $\mathcal{G}_q \subseteq \mathcal{G}$, and (c) a topological feature vector $\mathcal{F}_q$ as a proxy for query complexity (Obs. (5)). These lightweight signals guide budgeting in Phase 2 and scheduling in Phase 3 before LLM inference.

Phase 2: Adaptive Aggregation (Controller, Section *4.3).* Given $\mathcal{G}_q$, $\mathcal{F}_q$, and system backpressure S_{sys} (from the serving engine in Phase 3), this phase outputs budget K_q and prompt P_q with length L_q. It balances difficulty and load (Obs. (1)), allocating more context to hard queries and throttling easy ones under congestion, selecting K_i to maximize expected quality $Q(K_i, q_i)$.

Phase 3: SLO-Aware Scheduling (Actuator, Section *4.4).* Given $\{P_q\}$, SLOs $\{\tau_{q_i}\}$, and estimates ($\mathcal{F}_q$, S_{diff}, L_q), this phase outputs serving order π and results Y. It replaces FCFS with an SLO-aware priority queue that ranks queries by urgency and cost, mitigating HOL blocking (Obs. (2–3)) and maximizing QA-Goodput under **K**.

Note. (1) Phase 3 feeds congestion signals S_{sys} (*e.g.,* queue depth) back to Phase 2, enabling load-aware budget adjustment and joint optimization of **K** and π. (2) Sensor (priors), Controller (allocation), and Actuator (execution) together form a feedback-control system [20] that adapts to dynamic workloads without manual tuning.

Table 2: Summary of key notations.

Symbol	Description
$\mathcal{S}, \lambda$	Query batch and request arrival rate
$\mathcal{J}_\lambda$	QA-Goodput (Eq. 2)
$\mathbf{K}, \pi$	Budget vector and scheduling permutation (decision variables)
$\mathcal{G}$; q, $\mathcal{G}_q$	Knowledge graph; query and induced subgraph
$\mathcal{F}_q$	Topological feature vector of $\mathcal{G}_q$
$S_{\text{diff}}(q)$	Difficulty score $\in [0, 1]$ (Eq. 5)
K_q; $K_{\min}, K_{\max}, K_{\text{eff}}$	Query budget; lower, upper, and load-adjusted bounds
$u(t)$, θ_{bp}	Queue utilization and backpressure threshold
$L_i(K_i)$, μ_{p}	Prompt length and prefill throughput (Eq. 4)
τ	TTFT deadline (SLO)

4.2 Retrieval & Signal Extraction (The Sensor)

Overview. Given query q and corpus $\mathcal{D}$, this phase builds (if needed) an offline heterogeneous graph $\mathcal{G}$ with Document/Chunk/Entity nodes, induces a query subgraph $\mathcal{G}_q$ via two-hop structure-semantic retrieval (semantic anchoring + graph expansion), jointly re-ranks candidates into C_q, and extracts a lightweight topological feature vector $\mathcal{F}_q$ as structural priors for downstream budgeting and scheduling.

Heterogeneous graph construction. Given corpus $\mathcal{D}$, we construct (if not already available) an offline heterogeneous graph index $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$ (defined in Section 2.1) in three stages. First, each document $d \in \mathcal{D}$ is segmented into semantic chunks that preserve sentence boundaries [23]. Second, an offline LLM extracts entities and relational triples from each chunk via schema-constrained pipelines [12, 37, 55], producing canonicalized entity nodes grounded to a reference ontology (*e.g.,* Wikidata [46]). Third, we construct the heterogeneous graph by linking documents, chunks, and entities, forming inclusion edges (Document→Chunk→Entity) and relational edges (Entity↔Entity). This structure can expose *supernodes* (high-degree entities causing prompt variance; Obs. (2)) and *bridges* (cross-region connectors requiring larger K; Obs. (1)).

Structure-Semantic Collaborative Retrieval. Given q and $\mathcal{G}$, it induces a candidate subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ and a ranked candidate corpus C_q via a two-hop coarse-to-fine strategy (seed-expand) [19, 22, 42]: (1) *Hop-1 (semantic anchoring)* retrieves the top-K_1 chunks by bi-encoder cosine similarity to form C_q^{chunk}, collects their parent documents as C_q^{doc}, and traverses inclusion edges to gather mentioned entities as the seed set; (2) *Hop-2 (structure-guided expansion)* traverses up to two relational hops from the seeds to surface bridging evidence, then maps newly discovered entities back to their source chunks via inclusion edges to form C_q^{graph}, with Hop-1 and Hop-2 jointly constituting $\mathcal{G}_q$; and (3) *unified scoring* jointly re-scores $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ using a cross-encoder reranker to produce $s_{\text{rerank}}(d, q)$ and a single ranked list, while retaining bi-encoder embeddings $\mathbf{e}_d$ for Phase 2 (Section 4.3.3).

Topological signal extraction. We map $\mathcal{G}_q$ to features $\mathcal{F}_q \in \mathbb{R}^d$ as zero-cost structural priors for downstream control (Phases 2–3): they are inexpensive to compute and informative for initialization, though not used as standalone predictors. Let $\mathcal{E}(c)$ denote entities in chunk c, and $C_q^{\mathcal{G}}$ the chunk set. We introduce three key signals in $\mathcal{F}_q$ that will be used in later phases.

- *Bridge ratio* r_{bridge}: fraction of chunks sharing entities with ≥ 2 others; indicates cross-region connectivity and multi-hop complexity (Obs. 1), and provides the strongest single-feature signal.
- *Entity overlap density* ρ_{overlap}: fraction of chunk pairs sharing entities, capturing local connectivity and complementing r_{bridge}.
- *Entity count* n_{entity}: number of unique entities in $\mathcal{G}_q$, reflecting evidence breadth and reasoning depth.

Example 4: For the e-commerce support query q' ("payment succeeded but order failed …"), Phase 1 induces a subgraph $\mathcal{G}_{q'}$ that links evidence around *order*, *payment*, *inventory*, and *ticket*. It then extracts the corresponding feature vector $\mathcal{F}_{q'}$ (*e.g.,* flagging the "Error 503" entity as a supernode). The Controller (Phase 2) and Actuator (Phase 3) consume these signals to drive difficulty-aware budgeting and scheduling before the expensive prefill stage. □

Note. Extracting $\mathcal{F}_q$ involves only simple combinatorial operations over a small subgraph (typically < 500 nodes) and adds negligible overhead (typically < 5 ms in our implementation). This provides Phase 2 with an early signal that helps disentangle quality-driven budget needs from system-driven cost control.

4.3 Adaptive Aggregation (The Controller)

This subsection presents Phase 2, which acts as the *controller* between retrieval (Phase 1) and serving (Phase 3). It maps Phase 1 outputs $(\mathcal{G}_q, \mathcal{F}_q, C_q)$ to a query-specific budget K_q and an LLM-ready prompt under real-time backpressure.

We first establish a cost model linking K_q to prefill cost (Section 4.3.1), which underpins the budget control strategy. We then introduce a topology-based difficulty estimator and a backpressure-aware budget allocation method (Section 4.3.2). Finally, we describe a context assembler that selects and organizes K_q evidence units under the allocated budget (Section 4.3.3).

4.3.1 Cost Model.

To quantify how K_q affects serving cost and motivate adaptive budgeting, we model LLM inference cost. In autoregressive inference, each request incurs two sequential costs with distinct profiles [2, 61]. For query q_i with budget K_i, the end-to-end GPU occupancy is:

$$c_i(K_i, q_i) = \underbrace{L_i(K_i) \,/\, \mu_{\text{p}}}_{\text{prefill (compute-bound)}} + \underbrace{\hat{D}_i(q_i) \,/\, \mu_{\text{d}}}_{\text{decode (memory-bound)}}, \tag{4}$$

where $L_i(K_i) = |X_{\text{sys}}| + |q_i| + K_i \cdot \bar{\ell}$ is the prompt length, μ_{p} and μ_{d} are prefill and decode throughput (tokens/ms), and $\hat{D}_i(q_i)$ is the expected output length. The prefill term is *controllable*: K_i determines $L_i(K_i)$ and thus TTFT_i [26, 40]. The decode term is *observable but not controllable*, as $\hat{D}_i(q_i)$ depends on intrinsic query complexity.

Note. (1) This asymmetry motivates a two-lever design: Phase 2 controls prefill via K_i, while Phase 3 monitors queue utilization to handle combined prefill and decode effects. (2) The subgraph $\mathcal{G}_{q_i}$ from Phase 1 is informative for setting K_i: larger neighborhoods, higher branching, and fragmented structure require more evidence, motivating the topology-based difficulty estimator (Section 4.3.2).

4.3.2 Budget Allocation under Backpressure.

We introduce (a) a *cost-aware difficulty estimator* using Phase 1 topology signals, and (b) a *dynamic budget allocation* that maps difficulty and real-time utilization (Phase 3) to the final budget K_q.

Cost-aware difficulty estimator. To distinguish complex queries from simple ones, we define a scoring function $S_{\text{diff}} : \mathcal{Q} \to [0, 1]$ based on topology features extracted in Phase 1 (*i.e.,* bridge ra-

tio r_{bridge}, overlap density ρ_{overlap}, and entity count n_{entity}). Let $\tilde{\mathbf{f}}_q$ denote the min–max normalized feature vector; we compute

$$S_{\text{diff}}(q) = \mathbf{w}^\top \tilde{\mathbf{f}}_q, \tag{5}$$

where $\mathbf{w} \in \mathbb{R}^3$ is calibrated via grid search on held-out data. $S_{\text{diff}}(q) \approx 1$ indicates a hard query (*e.g.,* substantial cross-cluster bridging, moderate overlap, and many entities), whereas $S_{\text{diff}}(q) \approx 0$ indicates a simple lookup (*e.g.,* minimal bridging and low overlap).

Note. (1) S_{diff} is a *coarse continuous classifier* (hard vs. easy), not an oracle K^* regressor. (2) Features that separate difficulty (*e.g.,* r_{bridge}, ρ_{overlap}) do not localize gold evidence (which requires supervised selectors [15, 58], learned complexity models [24], or LLM profilers [40]); coarse ranking suffices as Phase 3 amplifies it into QA-Goodput gains. (3) S_{diff} proxies decode cost $\hat{D}_i(q_i)$ (Eq. 4), since more bridging typically induces longer reasoning chains [32]. (4) The estimator is *zero-cost and training-free* at serving time, reusing Phase 1 outputs with $\mathbf{w}$ calibrated via grid search on held-out data.

Dynamic budget allocation. Given $S_{\text{diff}}(q)$, the controller maps difficulty to budget K_q via a two-regime control law: (R1) maximize quality under low load, and (R2) degrade gracefully under congestion while preserving difficulty ordering.

We quantify load by queue utilization $u(t) \in [0, 1]$, *i.e.,* the ratio of pending prefill workload to the scheduling-window capacity:

$$u(t) = \Big(\sum_{i \in Q(t)} p_i\Big)/\delta, \qquad p_i = L_i(K_i)/\mu_{\text{p}}, \tag{6}$$

where $Q(t)$ is the set of pending/active prefill requests and δ is the scheduling window.

Let $\theta_{\text{bp}} \in [0, 1]$ be a backpressure threshold separating two regimes: full budgets when $u(t) \le \theta_{\text{bp}}$, and budget compression otherwise. This couples K_q with real-time load, reducing aggregate prefill cost while preserving relative prioritization.

Quality-first regime. If $u(t) \le \theta_{\text{bp}}$, K_q depends only on difficulty:

$$K_q = K_{\min} + \lfloor (K_{\max} - K_{\min}) \cdot S_{\text{diff}}(q) \rceil. \tag{7}$$

We assign larger budgets to harder queries and smaller ones to easier queries, using a linear map to keep K unimodal under Zipf workloads and avoid bimodality [20]. We denote this budget as $K_{\text{diff}}(q)$, also serves as candidate-pool size in context assembly.

Congestion regime. If $u(t) > \theta_{\text{bp}}$, apply a load-dependent cap:

$$K_{\text{eff}}(t) = \begin{cases} K_{\max}, & u(t) \le \theta_{\text{bp}}, \\ K_{\min} + (K_{\max} - K_{\min}) \cdot \phi(t), & \text{otherwise}, \end{cases} \tag{8}$$

where $\phi(t) = \exp\Big(-\alpha_{\text{bp}} \cdot \dfrac{u(t) - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\Big)$ and $\alpha_{\text{bp}} = \ln(K_{\max}/K_{\min})$.

Here the exponent $\phi(t)$ combines two normalized factors controlling budget contraction. The term $\frac{u(t)-\theta_{\text{bp}}}{1-\theta_{\text{bp}}} \in [0, 1]$ captures excess utilization (from congestion onset to saturation), while α_{bp} sets the compression range, ensuring $K_{\text{eff}}(t)$ decays from $K_{\max}$ to $K_{\min}$. Together, they yield exponential decay that is mild near the threshold and increasingly aggressive toward saturation.

Eq. (8) can be viewed as an online knapsack reservation function [51, 62], where exponential thresholds admit optimal competitive guarantees (see Appendix in [4] for the formal theorem and Section 5.4 for comparisons with alternative decay shapes). By construction, $K_{\text{eff}}(t)$ decreases monotonically with $u(t)$, smoothly interpolating from $K_{\max}$ to $K_{\min}$, while θ_{bp} defines a slack region that absorbs burstiness. Each query is then allocated proportionally within $[K_{\min}, K_{\text{eff}}(t)]$:

$$K_q = K_{\min} + \lfloor (K_{\text{eff}}(t) - K_{\min}) \cdot S_{\text{diff}}(q) \rceil, \tag{9}$$

which preserves relative difficulty while compressing the budget range. When $u(t) \le \theta_{\text{bp}}$, this reduces to Eq. (7).

Note. (1) Adaptive K and priority scheduling are complementary: Phase 2 controls prefill workload to induce cost heterogeneity ($K_{\text{easy}} \ll K_{\text{hard}}$), which Phase 3 exploits when ordering requests under SLO constraints. (2) The system operates in three regimes: (a) *quality-first* ($u(t) < \theta_{\text{bp}}$), (b) *transition* ($u(t) \approx \theta_{\text{bp}}$), and (c) *goodput-first* ($u(t) \gg \theta_{\text{bp}}$), where exponential decay dominates.

4.3.3 Context Assembler.

Given budget K_q, the *assembler* selects K_q evidence units from C_q and forms an LLM-ready prompt for Phase 3.

Objective. Construct a size-K_q set that (a) maximizes answer-critical coverage and (b) minimizes redundancy, jointly covering diverse regions of $\mathcal{G}_q$. We optimize a relevance-weighted coverage objective under a cardinality constraint via diversity-aware greedy selection.

Unified scoring and selection. For candidates $C_q = C_q^{\text{chunk}} \cup C_q^{\text{doc}} \cup C_q^{\text{graph}}$ (Phase 1), we deduplicate and assign each $d \in C_q$ a score:

$$s(d, q) = s_{\text{rerank}}(d, q) + \delta \cdot \mathbb{I}(d \in C_q^{\text{graph}}). \tag{10}$$

This yields a ranked list $\mathcal{D}_q$. If uncongested ($K_q = K_{\text{diff}}$), we apply top-K_q. Under backpressure ($K_q < K_{\text{diff}}$), we select K_q representatives from $\mathcal{D}_q[1{:}K_{\text{diff}}]$ via a relevance-weighted k-center objective:

$$\mathcal{S}_q^* = \arg\min_{|\mathcal{S}|=K_q} \max_{d \in \mathcal{D}_q[1:K_{\text{diff}}]} s(d, q) \cdot \big(1 - \max_{d' \in \mathcal{S}} \cos(\mathbf{e}_d, \mathbf{e}_{d'})\big), \tag{11}$$

where $\mathbf{e}_d$ denotes the embedding from Phase 1. The cosine term penalizes redundancy, while $s(d, q)$ preserves relevance. We solve Eq. (11) via greedy farthest-first traversal [17], achieving a 2-approximation in $O(|C_q| \cdot K_q)$ time (microseconds on CPU).

Context construction. We organize $\mathcal{S}_q^*$ by provenance and local proximity in $\mathcal{G}_q$, placing bridge entities near neighbors to support multi-hop reasoning. The final prompt is

$$P_q = [\, I_{\text{sys}},\ \mathcal{S}_q,\ q \,], \quad L_q = |P_q|. \tag{12}$$

with length $L_q = |P_q|$, passed to Phase 3 as the realized prefill cost.

Note. The assembler separates *how much* from *which*: topological signals determine global difficulty ($S_{\text{diff}} \to K_q$), while embedding-space coverage governs selection ($\mathcal{D}_q \to \mathcal{S}_q$). This preserves answer quality under constrained budgets, creates cost heterogeneity across queries, and enables Phase 3 to exploit it via SLO-aware scheduling.

4.4 SLO-Aware Scheduling (The Actuator)

Given budgets K_q from Phase 2, Phase 3 determines serving order π under SLO constraints to maximize QA-Goodput, acting as the *actuator* over shared GPU resources.

In the sequel, we begin by revisiting why FCFS breaks down under bursty arrivals and outlining the resulting challenges. We then present the scheduler's view, describing the signals it tracks and how they are combined into a priority score to address these challenges. Finally, we present a non-preemptive admission policy for GPU execution and the feedback loop through which signals

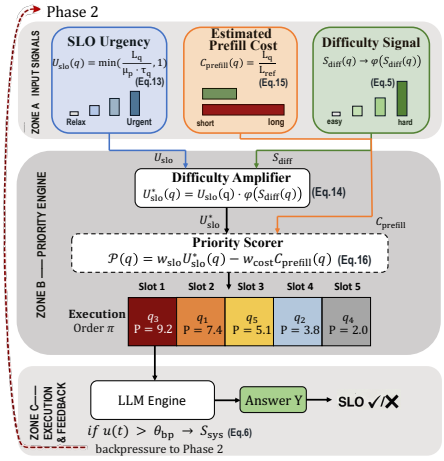


Figure 5: SLO-Aware Scheduling Logic. Priority $\mathcal{P}(q)$ combines deadline urgency U^*_{slo} (amplified by difficulty S_{diff} from Phase 2) and estimated prefill cost C_{prefill} to order the execution queue. A feedback monitor emits S_{sys} to Phase 2 when utilization exceeds θ_{bp}.

from Phase 3 inform subsequent budget decisions in Phase 2.

Motivation. Phase 2 produces queries $\{q\}$ with prompts $\{P_q\}$, each with prefill cost p_q and difficulty $S_{\text{diff}}(q)$. Under bursty arrivals, HOL blocking dominates: a few large-L_q queries can stall the queue and trigger cascading TTFT violations. We therefore replace FCFS with an SLO-aware priority scheduler balancing urgency and cost.

Challenges. Phase 3 faces three challenges: *(1) Cost heterogeneity*: adaptive budgets introduce substantial variance in prefill cost; *(2) Deadline awareness*: each query must satisfy its TTFT constraint (τ_q); and *(3) Queue coupling*: concurrent queries compete for shared GPU resources, exacerbating HOL blocking under bursty arrivals.

Scheduler view. We use an SLO-aware scheduler that ranks queries using signals from Phases 1–2. (1) We derive an urgency signal from each query's remaining TTFT slack, making the scheduler deadline-aware (Challenge 2). (2) We normalize urgency by the estimated prefill cost, preventing expensive queries from dominating and addressing cost heterogeneity (Challenge 1). (3) We use this urgency-per-cost ordering to mitigate queue coupling (Challenge 3) by reducing HOL blocking and interleaving short urgent jobs with longer ones. Finally, we incorporate a difficulty signal to further ensure fairness, preventing hard queries from being systematically deprioritized.

Priority signals. We adopt a lightweight scheduling score that integrates cost-model estimates with signals from Phase 2.

SLO urgency. We define the SLO urgency of a query q as

$$U_{\text{slo}}(q) = \min\left(p_q/\tau_q,\ 1\right), \tag{13}$$

where $p_q = L_q/\mu_{\text{p}}$ denotes the estimated prefill time and τ_q is the TTFT deadline. When $U_{\text{slo}}(q)$ is close to 1, it indicates that prefill alone nearly exhausts the SLO budget.

Difficulty-aware amplification. We further amplify urgency using difficulty score from Phase 2:

$$U^*_{\text{slo}}(q) = U_{\text{slo}}(q) \cdot \varphi(S_{\text{diff}}(q)), \tag{14}$$

where $\varphi(\cdot)$ is a monotone scaling function; higher difficulty increases the multiplier, boosting urgency and preventing hard queries from being systematically deprioritized.

Cost signal. To mitigate HOL blocking, we normalize prefill cost as

$$C_{\text{prefill}}(q) = L_q/L_{\text{ref}}, \tag{15}$$

where L_q is the prefill token length and L_{ref} is a fixed reference budget. Larger values indicate longer and more expensive prefills that are more likely to induce HOL blocking.

Priority function. We combine these signals into a unified score:

$$\mathcal{P}(q) = w_{\text{slo}} \cdot U^*_{\text{slo}}(q) - w_{\text{cost}} \cdot C_{\text{prefill}}(q), \tag{16}$$

where $w_{\text{slo}}, w_{\text{cost}} \geq 0$ are weights. The first term prioritizes urgent queries, while the second penalizes expensive ones, approximating urgency per unit cost with minimal overhead.

Scheduling policy. Given priority $\mathcal{P}(q)$, we adopt a priority-driven policy: at each step, the scheduler admits highest-$\mathcal{P}(q)$ queries to the next GPU batch (prefill), subject to resource constraints. This mitigates HOL blocking (vs. FCFS) and prevents large-K_q queries from monopolizing resources (vs. deadline-only scheduling).

System coupling. Phase 3 closes the loop with Phase 2 via utilization $u(t)$: when $u(t) > \theta_{\text{bp}}$, it feeds back to tighten budgets (Eqs. 8–9). Thus, Phase 2 controls per-query cost while Phase 3 controls execution order, enabling adaptation to dynamic workloads.

Starvation avoidance. Priority scheduling may starve low-priority queries. Under sustained congestion, backpressure (Eq. 8) compresses budgets toward $K_{\min}$, equalizing processing times and mitigating starvation; an optional aging mechanism can be adopted for strict fairness guarantees (see Appendix B.5 in [4] for details).

Note. (1) *Scheduling alone is insufficient*: with fixed budgets, even an optimal scheduler cannot reduce per-query cost. (2) *Budget adaptation alone is insufficient*: without scheduling, bursty arrivals still cause HOL blocking. (3) DynaGraph combines both: Phase 2 introduces cost heterogeneity, and Phase 3 exploits it via SLO-aware scheduling, jointly improving answer quality and SLO compliance.

4.5 Algorithm and Guarantee

In this subsection, we present the DynaGraph algorithm that integrates Phases 1–3 into a closed-loop control system, analyze its complexity, and establish its theoretical guarantees.

Algorithm. As shown in Algorithm 1, DynaGraph consists of three phases: (1) Phase 1 (*Sensor*, lines 1–3) initializes S_{sys}, constructs $\mathcal{G}$ from $\mathcal{D}$ (line 1), and, for each incoming query q, retrieves the subgraph $\mathcal{G}_q$ and features $\mathcal{F}_q$ (line 3); (2) Phase 2 (*Controller*, lines 4–11) maps $\mathcal{F}_q$ to $S_{\text{diff}} \in [0, 1]$ (line 4), computes the load-dependent ceiling K_{eff} from utilization $u(t)$ (lines 6–8), sets $K_q \in [K_{\min}, K_{\text{eff}}]$ (line 10), and assembles the prompt P_q with length L_q (line 11); (3) Phase 3 (*Actuator*, lines 12–15) amplifies urgency via S_{diff} (line 12), computes priority $\mathcal{P}(q)$ (line 13), enqueues q, and dispatches the highest-priority query q' for service, emitting its answer $Y_{q'}$ to the output stream (line 14); it then updates feedback by measuring $u(t)$ and emitting backpressure S_{sys} when $u(t) > \theta_{\text{bp}}$ (line 15).

Complexity analysis. Algorithm 1 takes $O(|V_q|+|E_q| + |C_q| \cdot K_q + \log n)$ per-query overhead (excluding LLM inference), where $O(|V_q|+|E_q|)$ is for subgraph retrieval and feature extraction (Phase 1), $O(|C_q| \cdot K_q)$ is for diversity-aware evidence selection

Algorithm 1: DynaGraph Closed-Loop Pipeline

Input : Query stream $\{q\}$, corpus $\mathcal{D}$, SLO deadlines $\{\tau_q\}$
Output: Answer stream $\{Y_q\}$

$S_{\text{sys}} \leftarrow 0$; $\mathcal{G} \leftarrow \text{Construct}(\mathcal{D})$
foreach *incoming query* q **do**
  /* Phase 1: Retrieval (Section 4.2) */
  $\mathcal{G}_q, \mathcal{F}_q, C_q \leftarrow \text{Retrieve}(q, \mathcal{G})$
  /* Phase 2: Adaptive Aggregation (Section 4.3) */
  $S_{\text{diff}} \leftarrow \mathbf{w}^\top \tilde{\mathbf{f}}_q$ // difficulty (Eq.5)
  if $u(t) > \theta_{\text{bp}}$ **then**
    $K_{\text{eff}} \leftarrow K_{\min} + (K_{\max} - K_{\min})\,\phi(t)$ // backpressure (Eq.8)
  else
    $K_{\text{eff}} \leftarrow K_{\max}$ // quality-first
  end
  $K_q \leftarrow K_{\min} + \lfloor (K_{\text{eff}} - K_{\min})\, S_{\text{diff}} \rceil$ // budget (Eq.9)
  $P_q, L_q \leftarrow \text{Assemble}(C_q, \mathcal{G}_q, K_q)$ // Selection (Eq.11)
  /* Phase 3: SLO-Aware Scheduling (Section 4.4) */
  $U^*_{\text{slo}} \leftarrow U_{\text{slo}} \cdot \varphi(S_{\text{diff}})$ // urgency (Eq.14)
  $\mathcal{P}(q) \leftarrow w_{\text{slo}} U^*_{\text{slo}} - w_{\text{cost}} C_{\text{prefill}}$ // priority (Eq.16)
  Insert q with key $\mathcal{P}(q)$; $Y_q \leftarrow \text{ServeHighestPriority}()$
  $u(t) \leftarrow \text{QueueUtilization}()$; $S_{\text{sys}} \leftarrow \mathbb{I}(u(t) > \theta_{\text{bp}})$
  // feedback (Eq.6)
end

from $|C_q|$ candidates (Phase 2), and $O(\log n)$ is for priority-queue insertion and dispatch among n concurrent queries (Phase 3).

Guarantee. We establish the following guarantee for the joint optimization formulation.

Theorem 2: *Let $\mathcal{J}(\mathbf{K}, \pi)$ denote the QA-Goodput under budget vector $\mathbf{K}$ and serving order π. Let $\mathbf{K}^{\text{fix}}$ be the static budget (K_i=$K_{\max}$ for all i), π^{FCFS} the arrival-order schedule, and $(\mathbf{K}^*, \pi^*)$ an optimal solution to the joint problem (Eq. 3) that can be obtained via Algorithm 1. Then,* $\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \max\big(\mathcal{J}(\mathbf{K}^*, \pi^{\text{FCFS}}),\ \mathcal{J}(\mathbf{K}^{\text{fix}}, \pi^*)\big)$ □

Proof. (1) By optimality of π^* for fixed $\mathbf{K}^*$, $\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^*, \pi^{\text{FCFS}})$. (2) Since $(\mathbf{K}^{\text{fix}}, \pi^*)$ is feasible for Eq. 3 and $(\mathbf{K}^*, \pi^*)$ is optimal, $\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^{\text{fix}}, \pi^*)$. □

Theorem 2 formalizes the complementarity observed in Table 1: neither adaptive budgets nor scheduling alone suffices, whereas an optimal joint solution dominates both. Additional results are provided in the Appendix B of [4], including scheduling approximation, queue stability, design rationale for exponential backpressure, and bounded starvation for DynaGraph and its components.

5 EXPERIMENTAL EVALUATION

Goal. We answer following questions via extensive experiments:

- **RQ1 (Overall performance):** Under concurrent serving load, how does DynaGraph compare to existing GraphRAG pipelines?
- **RQ2 (Serving strategy):** With retrieval fixed, how does DynaGraph's serving strategy compare with KV-cache-based backends, and what is the contribution of the closed-loop interaction between Phase 2 and Phase 3?
- **RQ3 (Design choice ablation):** How do core design choices in adaptive aggregation (*e.g.,* budget-decay shape and topological-feature selection) affect the quality–latency trade-off?
- **RQ4 (Robustness):** How robust is DynaGraph to changes in request load and deployment knobs (*e.g.,* $K_{\min}$ and θ_{bp})?
- **RQ5 (Scalability):** In a fully live retrieval pipeline, does DynaGraph remain effective, and does it generalize across LLMs and corpus scales?

Explanation. (1) RQ1 establishes overall performance by comparing DynaGraph with representative GraphRAG pipelines under concurrent load. (2) RQ2 isolates the serving layer by fixing retrieval (*i.e.,* treating candidate evidence C_q as given) and comparing against KV-cache-based backends, quantifying the synergy benefits of adaptive budgeting and scheduling. (3) RQ3 further ablates Phase 2, which maps graph structure and runtime load to per-query budgets, to justify key design choices (*e.g.,* the exponential decay in Eq. (8)). (4) RQ4 evaluates robustness of DynaGraph under different workload intensity and system parameters. (5) RQ5 evaluates a fully live pipeline with online retrieval (embedding and reranking), testing end-to-end effectiveness under realistic deployment conditions.

Evaluation Protocol. (1) RQ1–RQ4 evaluate DynaGraph under a serving-only setting with pre-computed retrieval to isolate serving-layer effects: the candidate set C_q is fixed in advance, and only budget allocation, ranking, and/or scheduling are varied. This allows us to attribute performance differences to Phase 2 and Phase 3 without confounding retrieval quality. (2) RQ5 evaluates DynaGraph under a fully live end-to-end setting with online retrieval (*e.g.,* graph construction, embedding computation, and reranking), capturing the complete pipeline cost.

We further study context selection (RQ6) and phase ablation (RQ7); see Appendix D in [4] for details.

5.1 Setup

Datasets. We evaluate on three established multi-hop QA benchmarks: 2WikiMultihopQA [21] (2Wiki; 12,576 queries; multi-path, bridge-entity reasoning), HotpotQA [52] (7,405 queries; cross-document reasoning), and MuSiQue [44] (2,417 queries; compositional reasoning up to 4 hops). Following the protocol of HippoRAG [19] and subsequent works [22, 42, 57], we sample 1,000 queries from each validation set, using their associated supporting and distractor passages as the retrieval corpus $\mathcal{D}$ (approximately 11K passages per dataset). For scalability analysis (RQ5), we additionally evaluate on the full 2Wiki validation set (~57K passages).

Models. We use two primary LLMs spanning distinct architectural families: Qwen3-4B [49] (pure attention; single RTX 4090) and Qwen3.5-4B [38] (hybrid Mamba; 2×RTX 4090 with tensor parallelism, TP=2). We additionally include Qwen2.5-7B [50] in RQ2 to evaluate performance under moderate serving load. For retrieval, we use Qwen3-Embedding-0.6B [59] for dense encoding and bge-reranker-v2-m3 [11] for cross-encoder reranking.

Infrastructure. All experiments are conducted on 2×NVIDIA RTX 4090 GPUs (24 GB each) with 128 GB of RAM. We use vLLM [30] for LLM serving. Unless otherwise specified for a given RQ, workloads follow the Zipfian popularity model from Section 3.1 (α=1.0, λ=30 QPS; 3,000 total requests). Please see Appendix C in [4] for more infrastructure details.

Metrics. We evaluate along four complementary dimensions: (1) *Answer quality*, measured by token-level F_1 on deduplicated unique queries to avoid popularity bias under Zipf replay [44, 52]; (2) *Serv-*

Table 3: End-to-end system comparison under online serving. Qwen3.5-4B, 1000 unique queries, Zipf@30 qps, 3000 total requests. $\mathcal{J}=\frac{1}{N}\sum_i F_1^{(i)}\cdot\mathbb{I}[\mathbf{TTFT}_i \le \tau]$.

Method	F1	Acc	Toks	p50 (ms)	SLO@500	SLO@200	$\mathcal{J}$@500	$\mathcal{J}$@200
2WikiMultihopQA								
SiReRAG	.721	.728	6760	8629	0.0%	0.0%	0.0%	0.0%
KET-RAG	.470	.438	11833	19255	0.0%	0.0%	0.0%	0.0%
ClueRAG	.637	.644	1165	788	6.8%	0.1%	4.8%	0.1%
HippoRAG	.601	.673	961	563	34.8%	0.2%	22.1%	0.1%
DynaGraph	**.787**	**.841**	2391	**220**	**94.9%**	**45.5%**	**72.8%**	**37.5%**
HotpotQA								
SiReRAG	.570	.483	5482	6346	0.0%	0.0%	0.0%	0.0%
KET-RAG	.603	.524	11613	19144	0.0%	0.0%	0.0%	0.0%
ClueRAG	.512	.455	709	725	5.8%	0.1%	3.3%	0.0%
HippoRAG	.499	.535	1202	509	47.2%	1.3%	21.9%	0.7%
DynaGraph	**.769**	**.716**	2244	**224**	**95.5%**	**40.3%**	**53.3%**	**21.5%**
MuSiQue								
SiReRAG	.427	.266	6712	7751	0.0%	0.0%	0.0%	0.0%
KET-RAG	.301	.152	11909	19060	0.0%	0.0%	0.0%	0.0%
ClueRAG	.409	.250	904	715	6.8%	0.2%	3.1%	0.0%
HippoRAG	.271	.265	1282	481	53.9%	1.0%	14.8%	0.1%
DynaGraph	**.471**	**.507**	2088	**173**	**97.7%**	**63.6%**	**47.6%**	**32.8%**

ing latency, measured by Time-to-First-Token (TTFT) at p50/p95, the primary latency metric in LLM serving systems [30, 61]; (3) *Service reliability*, measured by SLO attainment, *i.e.,* the fraction of requests completed within a deadline τ [2, 61]; and (4) *QA-Goodput*, $\mathcal{J}_\lambda$ (Eq. 2), which weights each on-time request by its F_1 score, jointly capturing accuracy and timeliness in a single metric. $\mathcal{J}_\lambda$ is computed per-request to reflect actual serving conditions. We report $\mathcal{J}_\lambda$ at $\tau \in \{200, 500, 2000\}$ ms to cover strict to relaxed SLO regimes; all latency values are in ms unless stated otherwise.

Baselines. We implement DynaGraph in Python and compare against the following baselines with released code: (1) *GraphRAG-based methods:* (a) SiReRAG [57] (dual hierarchical indexing), (b) KET-RAG [22] (KG skeleton with a bipartite keyword–entity graph), (c) ClueRAG [42] (query-driven iterative traversal with reranking), and (d) HippoRAG [19] (lightweight PPR-based graph propagation); (2) *KV-cache-based serving methods* (for RQ2): (e) CacheBlend [53] (chunk-level KV offloading), and (f) DepCache [54] (sparse dependency attention for graph-path prefix sharing); and (3) *a DynaGraph variant:* (g) VanillaGRAG, which directly uses the Phase 1 output without the optimizations in Phases 2 and 3. All baselines use the same LLM backend and serving conditions for fair comparison.

5.2 RQ1: Overall Performance

We compare DynaGraph against 4 representative GraphRAG pipelines (SiReRAG, KET-RAG, ClueRAG and HippoRAG) on tested datasets. As shown in Table 3, our key findings are as below.

(1) DynaGraph consistently achieves the best answer quality across all datasets (F_1: 0.787/0.769/0.471), outperforming the strongest baseline by 9–74%. The gain is most pronounced on MuSiQue (+0.200 over HippoRAG, +0.062 over ClueRAG), where multi-hop compositional reasoning requires bridging evidence across multiple entity clusters. This is because DynaGraph preserves such dispersed evidence via coverage-aware assembly (Section 4.3.3), whereas single-strategy retrievers either discard it (HippoRAG) or fail to prioritize it (ClueRAG).

(2) DynaGraph also achieves the lowest median TTFT (220/224/173 ms), despite using longer contexts on average than lightweight baselines such as HippoRAG and ClueRAG. In contrast, SiReRAG and KET-RAG forward large fixed contexts (5–12k tokens), leading to extreme latency (p50 up to 19.3 s) and complete SLO failure. ClueRAG and HippoRAG use moderate context sizes but lack scheduling coordination, resulting in higher latency (481–788 ms) and lower SLO attainment (6.8–53.9%). This is because DynaGraph assigns small contexts to most easy queries via adaptive budgeting ($K \le 5$, ~500 tokens), thereby reducing the median prefill cost, while SLO-aware scheduling prioritizes low-cost queries, allowing them to clear the queue quickly.

(3) DynaGraph further achieves the best *QA-Goodput*, *e.g.,* it reaches $\mathcal{J}$@500 of 72.8%/53.3%/47.6%, outperforming HippoRAG by 2.4×–3.3×. Under stricter SLO (τ=200 ms), the gap widens further (37.5%/21.5%/32.8% vs. all baselines < 1.3%). This confirms that DynaGraph does not merely reorder execution, but jointly optimizes evidence budgeting and scheduling to maximize on-time and high-quality responses.

These results validate that joint optimization is essential to reconcile answer quality with serving latency, and highlight the effectiveness of the proposed closed-loop design.

5.3 RQ2: Serving Strategy Comparison

We evaluate the serving strategy enabled by the closed-loop interaction between Phases 2 and 3 by isolating the serving layer. Specifically, we fix the retriever (Phase 1) and compare DynaGraph against variants that replace Phases 2 and 3 with KV-cache-based methods (CacheBlend and DepCache) or a no-optimization baseline (VanillaGRAG). For fair comparison, all methods receive the same Phase 1 retrieval output (~30k tokens).

As shown in Table 4: (1) DynaGraph consistently achieves the best QA-Goodput across both Qwen3-4B and Qwen2.5-7B, reaching the highest $\mathcal{J}$@500 (78.6%/64.8%). Notably, it matches the lowest latency of DepCache (p50: 143 vs. 141 ms on Qwen3-4B), while delivering 2.4×/1.4× higher F1, demonstrating a better quality–latency trade-off. (2) VanillaGRAG establishes the cost upper bound by directly forwarding the full Phase 1 output (~30k tokens), leading to prohibitive latency (p50: 45 s on Qwen3-4B and 4.9 s on Qwen2.5-7B) and near-zero QA-Goodput ($\mathcal{J}$@500 ≤ 1.4%). This confirms that controlling prompt length is necessary for any practical serving system. (3) CacheBlend improves prefill throughput via per-chunk KV reuse but does not reduce prompt length, resulting in high latency (p50: 18.3 s / 11.0 s) and 0% SLO@500. Moreover, it fails to preserve cross-document dependencies inherent in GraphRAG, reducing F1 to 0.667 (−14%). This shows that improving μ_p alone cannot compensate for an oversized context. (4) DepCache achieves excellent latency (p50: 141 ms, 99.5% SLO@500) but at the cost of answer quality (F1: 0.308/0.427, $\mathcal{J}$@500: 30.7%/41.2%). By restricting attention to graph-structured token pairs, it cannot fully model document- and chunk-level evidence, leading to incomplete reasoning. This highlights a fundamental limitation: reducing computation via sparse attention can break multi-hop evidence integration.

The advantage of DynaGraph lies in reducing prompt length *before* inference via adaptive K, while preserving all evidence types under full attention. This yields the highest F_1 (0.752/0.602) and $\mathcal{J}$@200 (61.1%/47.5%) across both models. These results validate that only the closed-loop interaction between Phases 2 and 3 can

Table 4: Serving baseline comparison with fixed retriever. 2Wiki, 1000q×3000, Zipf@30 qps.

Method	F1↑	Acc↑	p50 (ms)↓	p95 (ms)↓	SLO@500↑	$\mathcal{J}$@500↑	$\mathcal{J}$@200↑
Qwen3-4B							
VanillaGRAG	.777	.817	45053	85912	0.2%	0.2%	0.0%
CacheBlend [53]	.667	.725	18298	34090	0.0%	0.0%	0.0%
DepCache [54]	.308	.466	**141**	344	**99.5%**	30.7%	21.3%
DynaGraph	**.752**	**.797**	143	**339**	97.9%	**78.6%**	**61.1%**
Qwen2.5-7B							
VanillaGRAG	.555	.587	4885	10827	2.8%	1.4%	0.7%
CacheBlend [53]	.564	.595	11035	12655	0.0%	0.0%	0.0%
DepCache [54]	.427	.457	213	467	**96.3%**	41.2%	19.8%
DynaGraph	**.602**	**.621**	**144**	**433**	96.2%	**64.8%**	**47.5%**

jointly control cost and execution order, enabling consistent gains in QA-Goodput under realistic serving constraints.

We further ablate the contributions of Phases 2 and 3 individually on Qwen3.5-4B (see Appendix D.2 in full version [4] for details). We find that neither phase alone can achieve both high SLO attainment and high QA-Goodput, consistent with the observations in Section 3.2. This further justifies the need for our joint design that couples adaptive budgeting (Phase 2) with SLO-aware scheduling.

5.4 RQ3: Design Choice Ablation

In this subsection, we analyze key design choices in the controller of DynaGraph, the most critical component of the system. Specifically, we study (1) the decay shape of the budget ceiling $K_{\text{eff}}(t)$ under congestion (Eq. 8) and (2) the cost-aware difficulty estimator $S_{\text{diff}}(q)$ (Eq. 5). Experiments are conducted on 2Wiki with Qwen3.5-4B under Zipf@30 QPS (Figure 6).

Impact of decay shape of $K_{eff}(t)$. As shown in Figure 6(a), (a) DynaGraph with exponential decay (Eq. 8) achieves the best overall quality–latency trade-off among all strategies (Linear, Fixed, Threshold). It attains the highest F1 (0.767) and $\mathcal{J}$@2s (75.4%) by retaining a larger average budget (avg K=12.5, +24% over Linear), thereby preserving context for hard queries under sustained load. (b) While the linear variant performs slightly better under strict SLO ($\mathcal{J}$@200: 37.8% vs. 35.7%), it over-compresses budgets and reduces F1 (0.761), weakening difficulty differentiation. (c) The fixed strategy fails to absorb load spikes (p99 up to 15,687 ms), and the threshold strategy collapses all queries to $K_{\min}$, causing a large F1 drop (0.605). These results show that a smooth, load-aware decay is necessary to balance quality and latency across SLO regimes.

We further examine its interaction with backpressure from Phase 3 (Figure 6(b)). (a) Exponential decay reduces avg K from 20.0 to 12.5 (−37.5%) while preserving clear separation between hard and easy queries (83% vs. 72.5% of $K_{\max}$), with no queries collapsed to $K_{\min}$. (b) In contrast, linear and threshold strategies compress all queries to similar budgets, eliminating the difficulty signal. This pattern generalizes across datasets (avg K≈12.4–12.6) while maintaining high SLO@2s (≥98.6%). These results highlight the robustness of exponential decay under dynamic backpressure.

Impact of difficulty estimator with topological features. As shown in Figure 6(c), (a) combining the three topological signals from Phase 1 (r_{bridge}, ρ_{overlap}, and n_{entity}) achieves the best performance, maintaining AUROC ≈ 0.80 from 1K to 12.6K queries. (b) Individual features exhibit complementary trends: r_{bridge} and ρ_{overlap} slightly decrease as graph density increases, while n_{entity} becomes more discriminative; their combination remains stable across scales. (c) Adding an additional feature $d_{\max}$ yields less than 1 pp improvement, indicating diminishing returns. This confirms that the three selected features are sufficient and effective for difficulty estimation.

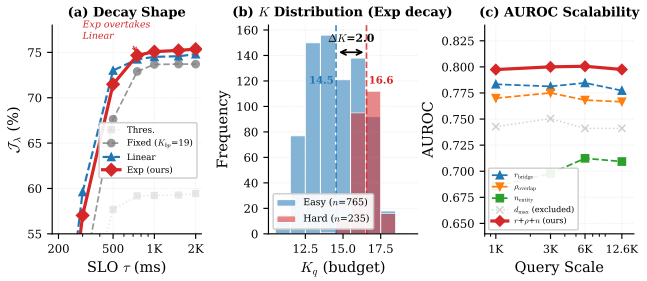


Figure 6: Phase 2 design ablation (2Wiki, Qwen3.5-4B, λ=30). (a) Exp decay achieves the best F1 while bounding tail latency. (b) Hard/Easy K separation under backpressure. (c) Three features (r+ρ+n) achieve stable AUROC ≈ 0.80; $d_{\max}$ excluded.

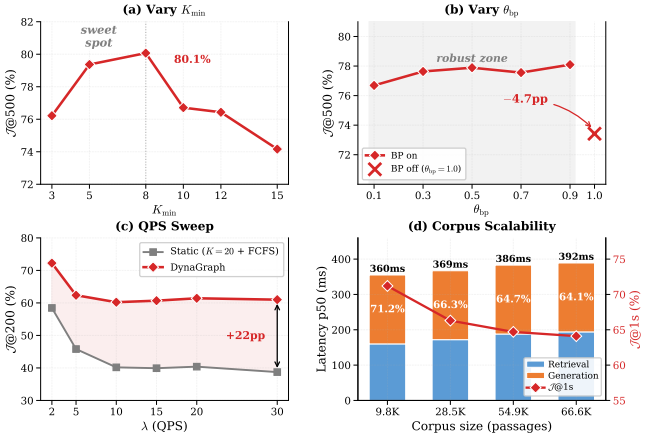


Figure 7: Robustness. (a) $K_{\min}$ sweep. (b) θ_{bp} sweep; × marks BP disabled. (c) DynaGraph vs Static under increasing load. (d) E2E latency breakdown and $\mathcal{J}$@1s under corpus growth.

5.5 RQ4: Robustness

We evaluate the robustness of DynaGraph along three dimensions: (1) sensitivity to the minimum size for query budget K, (2) sensitivity to backpressure activation threshold θ_{bp}, and (3) sensitivity under increasing request load ratio λ.

Varying minimum budget size $K_{\min}$. We evaluate the performance of DynaGraph by varying $K_{\min}$ from 3–15. As shown in Figure 7(a), performance improves as $K_{\min}$ increases from 3 to 8, reaching a peak $\mathcal{J}\lambda$@500 of 80.1%, and then gradually declines for larger values. This is because a small Kmin under-allocates evidence for hard queries, while an overly large $K_{\min}$ reduces flexibility and pushes latency toward the SLO boundary. We find that DynaGraph performs well when $K_{\min} \in [5, 10]$.

Varying backpressure activation threshold θ_{bp}. We evaluate the per-

Table 5: Cross-model generalization (2WikiMultihopQA, DynaGraph, 1000 unique queries, 3000 Zipf requests, 2×RTX 4090).

Model	λ	F1↑	Acc↑	p50↓ (ms)	p95↓ (ms)	SLO@500↑	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑	$\mathcal{J}$@1s↑
Qwen3.5-0.8B	200	.552	.587	84	155	100.0%	49.0%	49.4%	49.4%
Qwen3.5-2B	100	.627	.694	91	202	100.0%	**71.9%**	75.4%	75.4%
Qwen3.5-4B	100	.789	.851	201	430	98.1%	44.0%	83.1%	84.5%
Llama-3-8B	80	.599	.619	147	440	96.4%	48.3%	66.1%	67.0%
Ministral-8B	100	.749	.797	122	372	97.4%	62.0%	78.5%	78.7%
Qwen3.5-9B	80	.791	.831	321	744	79.7%	27.6%	68.0%	83.2%
Qwen3.6-35B	2	**.822**	**.874**	265	587	90.7%	21.0%	78.0%	**84.2%**

formance of DynaGraph by varying θ_{bp} from 0.1–1.0. As shown in Figure 7(b), performance remains stable when $\theta_{\text{bp}} \in [0.1, 0.9]$, with $\mathcal{J}\lambda$@500 varying within 1.4%, and drops by 4.4pp when θbp = 1.0. This is because moderate θ_{bp} enables timely activation of backpressure under congestion, whereas disabling it reducesrates to a static allocation that fails under strict SLOs. We find that DynaGraph is robust to θ_{bp} within a wide range.

Varying request load ratio λ. We evaluate the performance of DynaGraph by varying λ from 2–30. As shown in Figure 7(c), DynaGraph maintains stable performance (60–72% $\mathcal{J}_\lambda$@200), while the static baseline (i.e., the phase-silo configuration in Section 3.1) drops significantly from 59% to 39%. This is because difficulty-aware budget allocation provides a strong baseline, while backpressure further mitigates congestion as load increases. We observe that DynaGraph remains stable under increasing workload intensity.

5.6 RQ5: Scalability

We evaluate the scalability of DynaGraph along two key dimensions: (1) graph and query scalability under corpus growth and fully live pipelines, and (2) model scalability across different LLM sizes. Results are shown in Figure 7 and Table 5.

Graph and query scalability. As shown in Figure 7(d), DynaGraph scales effectively with increasing corpus size and maintains strong performance under fully live end-to-end pipelines. (1) As the corpus grows from 9.8K to 66.6K passages (7×), end-to-end p50 latency increases by only 9% (360→392 ms), while $\mathcal{J}$@1s decreases moderately (71.2%→64.1%). (2) Under fully online pipelines, the F_1 of DynaGraph remains comparable to the pre-computed setting (*e.g.,* 2Wiki: 0.801 vs. 0.787). This scalability stems from the design of DynaGraph: Phase 1 bounds retrieval complexity via two-hop expansion and fixed-size reranking, while Phase 2 operates on a fixed-size candidate set, making the overall cost largely insensitive to corpus growth. This indicates that DynaGraph scales to larger corpora without SLO violations, and its fully dynamic retrieval pipeline achieves competitive performance under evolving data.

Model scalability. As shown in Table 5, DynaGraph generalizes across model families and sizes without tuning and adapts effectively to different SLO regimes: (1) F_1 increases with model size within the Qwen family (0.552→0.822) while maintaining strong $\mathcal{J}$ across all models; and (2) the optimal model shifts with the latency budget—smaller models (2B) dominate under strict SLO ($\mathcal{J}$@200: 71.9%), medium models (4B) under moderate SLO, and larger models (9B/35B) under relaxed SLO. This reflects the inherent trade-off that larger models improve quality at the cost of higher latency; DynaGraph preserves this trade-off while ensuring each model operates near its Pareto-optimal point via adaptive budgeting and backpressure. Portability beyond Qwen is confirmed by Ministral-8B ($\mathcal{J}$@500: 78.5%) and Llama-3-8B ($\mathcal{J}$@500: 66.1%), where the serving layer operates correctly (SLO@500 ≥ 96%) regardless of the underlying LLM's reasoning capability.

6 RELATED WORK

We categorize existing work into three groups.

Graph-based retrieval for multi-hop QA. Hierarchical retrieval methods such as RAPTOR [41] and GraphRAG [16] organize corpora into trees or communities to surface long-range evidence. Entity- and relation-centric approaches, including HippoRAG [19], SiReRAG [57], KET-RAG [22], and Clue-RAG [42], build explicit graph indices to support multi-hop expansion, such as dual-perspective indexing, multi-granular graphs, and query-driven iterative traversal. While these systems improve evidence coverage and reasoning faithfulness, they typically assume fixed retrieval budgets and do not explicitly account for serving-time latency or throughput constraints.

RAG serving and caching. General LLM serving systems, such as vLLM [30], SGLang [60], and DistServe [61], improve memory management and scheduling but remain largely retrieval-agnostic. RAG-oriented systems, including RAGO [26], METIS [40], RAGCache [27], CacheBlend [53], and DepCache [54], reduce prefill overhead through configuration adaptation or KV-cache reuse. However, these techniques primarily optimize execution after evidence selection, rather than controlling the evidence budget itself or adapting retrieval to instantaneous load.

Adaptive context selection. Adaptive-RAG [24] and Self-RAG [5] route queries based on estimated complexity. RECOMP [48] and LongLLMLingua [25] compress prompts after retrieval. Adaptive-k [43] and CompSelect [58] predict query-specific budgets. These works show that fixed-K prompting is often wasteful, but they mainly rely on textual similarity or learned surrogates and are rarely integrated with serving backpressure.

How DynaGraph differs. DynaGraph bridges retrieval, budget selection, and serving. It uses lightweight graph-structural signals to estimate query difficulty, proactively controls the amount of evidence rather than only compressing or caching after retrieval, and couples this control with load-aware scheduling to jointly optimize answer quality and timeliness.

7 CONCLUSION

We present DynaGraph, a closed-loop GraphRAG framework that jointly optimizes evidence selection and serving under SLO constraints. Built on the proposed QA-Goodput metric, our preliminary analysis reveals that fixed-budget retrieval and phase-isolated pipelines break down under cost heterogeneity and bursty arrivals, as they fail to align retrieval decisions with serving-time dynamics. DynaGraph addresses this limitation through a unified, feedback-driven design: (a) a topology-based difficulty estimator that captures query complexity, (b) a backpressure-aware budget controller that adapts context under system load, and (c) an SLO-aware scheduler that prioritizes queries based on urgency and estimated cost.

REFERENCES

[1] Shubham Agarwal, Sai Sundaresan, et al. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data(SIGMOD)* 3, 3 (2025), 1–28.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[3] Aisha Alansari and Hamzah Luqman. 2026. Large language models hallucination: A comprehensive survey. *Computer Science Review* 61 (2026), 100970.

[4] Anonymous. 2026. DynaGraph: Code, Artifacts, and Full Version. https://anonymous.4open.science/r/DynaGraph-Releasee-0496

[5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[6] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems.* " O'Reilly Media, Inc.".

[7] Angela Bonifati, Wim Martens, John Timm, et al. 2017. An Analytical Study of Large SPARQL Query Logs. *Proceedings of the VLDB Endowment* 11, 2 (2017), 149–161.

[8] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. 1999. Web caching and Zipf-like distributions: Evidence and implications. In *Proceedings of IEEE INFOCOM*. IEEE.

[9] Niv Buchbinder and Joseph Naor. 2009. *The design of competitive online algorithms via a primal-dual approach.* Now Publishers.

[10] Deeparnab Chakrabarty, Yunhong Zhou, and Rajan Lukose. 2008. Online knapsack problems. In *Workshop on Internet and Network Economics (WINE)*.

[11] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL]

[12] Alla Chepurova, Aydar Bulatov, Mikhail Burtsev, and Yuri Kuratov. 2026. Wikontic: Constructing Wikidata-Aligned, Ontology-Aware Knowledge Graphs with Large Language Models. In *EACL*. 8304–8319. https://doi.org/10.18653/v1/2026.eacl-long.388

[13] Yingnong Dang, Qingwei Lin, and Peng Huang. 2019. AIOps: Real-World Challenges and Research Innovations. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion (ICSE-Companion)*. 4–5.

[14] George B. Dantzig. 1957. Discrete-Variable Extremum Problems. *Operations Research* 5, 2 (1957), 266–288.

[15] Jiale Deng, Yanyan Shen, Ziyuan Pei, Youmin Chen, and Linpeng Huang. 2025. Influence Guided Context Selection for Effective Retrieval-Augmented Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[16] Darren Edge et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

[17] Teofilo F. Gonzalez. 1985. Clustering to Minimize the Maximum Intercluster Distance. *Theoretical Computer Science* 38 (1985), 293–306.

[18] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[19] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Joseph L Hellerstein, Yixin Diao, Sujay Parekh, and Dawn M Tilbury. 2004. *Feedback control of computing systems.* John Wiley & Sons.

[21] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *COLING*.

[22] Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. Ket-rag: A cost-efficient multi-granular indexing framework for graph-rag. In *KDD*. 1003–1012.

[23] Isaacus. 2025. semchunk: A fast, lightweight and easy-to-use Python library for splitting text into semantically meaningful chunks. https://github.com/isaacus-dev/semchunk

[24] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 7036–7050.

[25] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[26] Wenqi Jiang et al. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. In *ISCA*.

[27] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems* 44, 1 (2025), 1–27.

[28] Kostis Kaffes, Timothy Chong, Jack T. Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. 2019. Shinjuku: Preemptive Scheduling for μsecond-scale Tail Latency. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[29] Richard M. Karp. 1972. Reducibility among Combinatorial Problems. In *Complexity of Computer Computations*. Plenum Press, 85–103.

[30] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.

[31] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[32] Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, and Mingming Liu. 2025. Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems. *arXiv preprint arXiv:2510.24476* (2025).

[33] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, Feifei Li, Changcheng Chen, and Dan Pei. 2020. Diagnosing Root Causes of Intermittent Slow Queries in Cloud Databases. *Proceedings of the VLDB Endowment (PVLDB)* 13, 8 (2020), 1176–1189. https://doi.org/10.14778/3389133.3389136

[34] Costas Mavromatis and George Karypis. 2024. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. *arXiv preprint arXiv:2405.20139* (2024).

[35] Sean P. Meyn and Richard L. Tweedie. 2009. *Markov Chains and Stochastic Stability* (2nd ed.). Cambridge University Press.

[36] Michael Neely. 2010. *Stochastic network optimization with application to communication and queueing systems.* Morgan & Claypool Publishers.

[37] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[38] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[39] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2383–2392.

[40] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 606–622.

[41] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[42] Yaodong Su, Yixiang Fang, Yingli Zhou, Quanqing Xu, and Chuanhui Yang. 2026. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. In *ICDE*.

[43] Chihiro Taguchi, Seiji Maekawa, and Nikita Bhutani. 2025. Efficient Context Selection for Long-Context QA: No Tuning, No Iteration, Just Adaptive-k. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[44] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single Hop Question Composition. *TACL* (2022).

[45] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *ACL*. Association for Computational Linguistics, Toronto, Canada, 10014–10037. https://doi.org/10.18653/v1/2023.acl-long.557

[46] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[47] Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxin Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, et al. 2026. From Prefix Cache to Fusion RAG Cache: Accelerating LLM Inference in Retrieval-Augmented Generation. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–28.

[48] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[50] An Yang, Baosong Yang, et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[51] Lei Yang, Mohammad Hajiesmaili, Ramesh Sitaraman, and Don Towsley. 2022. Competitive algorithms for online multidimensional knapsack problems. In *Proc. ACM SIGMETRICS*.

[52] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.

[53] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang,

Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[54] Hao Yuan et al. 2026. DepCache: A KV Cache Management Framework for GraphRAG with Dependency Attention. In *Proceedings of the ACM on Management of Data (SIGMOD)*.

[55] Bowen Zhang and Harold Soh. 2024. Extract, Define, Canonicalize: An LLM-Based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9820–9836.

[56] Liangliang Zhang, Zhuorui Jiang, Hongliang Chi, Haoyang Chen, Mohammed ElKoumy, Fali Wang, Qiong Wu, Zhengyi Zhou, Shirui Pan, Suhang Wang, et al. 2025. Diagnosing and Addressing Pitfalls in KG-RAG Datasets: Toward More Reliable Benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[57] Nan Zhang, Prafulla Kumar Choubey, Alexander Fabbri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. 2025. SiReRAG: Indexing Similar and Related Information for Multihop Reasoning. In *ICLR*.

[58] Qianchi Zhang, Hainan Zhang, Liang Pang, Yongxin Tong, Hongwei Zheng, and Zhiming Zheng. 2026. Less is More: Compact Clue Selection for Efficient Retrieval-Augmented Generation Reasoning. In *Proceedings of the ACM Web Conference 2026 (WWW)*.

[59] Yanzhao Zhang, Mingxin Li, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).

[60] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*.

[61] Yinmin Zhong, Shengyu Liu, Jianbo Chen, et al. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[62] Yunhong Zhou, Deeparnab Chakrabarty, and Rajan Lukose. 2008. Budget constrained bidding in keyword auctions and online knapsack problems. In *Proceedings of the 17th international conference on world wide web*. 1243–1244.

Table 6: Summary of Key Notations.

Symbol	Description
	Metric & Problem (Section 2.2)
$\mathcal{S} = \{q_i\}_{i=1}^n$	Query batch (n queries)
λ	Request arrival rate
τ_{q_i}	TTFT deadline for query q_i
$\text{Score}(q_i)$	Answer quality $\in [0,1]$ (e.g., token-level F1)
$\mathcal{J}_\lambda$	QA-Goodput (Eq. 2)
$Q(K_i, q_i)$	Expected answer quality $\mathbb{E}[\text{Score}(q_i) \mid K_i]$
$\mathbf{K} = (K_1, \ldots, K_n)$	Per-query budget vector (decision variable)
π	Serving order / scheduling permutation (decision variable)
	Graph & Retrieval (Phase 1)
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$	Global heterogeneous knowledge graph
q, $\mathcal{G}_q$	User query and its induced candidate subgraph
$\mathcal{D}$	Raw document corpus (offline input)
$\mathcal{F}_q$	Topological feature vector of $\mathcal{G}_q$ (Sensor signal)
r_{bridge}	Bridge ratio: fraction of chunks connecting ≥ 2 entity clusters
ρ_{overlap}	Entity overlap density: fraction of chunk pairs sharing ≥ 1 entity
n_{entity}	Number of unique entities in $\mathcal{G}_q$
	Adaptive Aggregation (Phase 2)
$S_{\text{diff}}(q)$	Normalized difficulty score $\in [0,1]$ (Eq. 5)
S_{sys}	Binary backpressure gate: $\mathbb{I}(u(t) > \theta_{\text{bp}})$
θ_{bp}	Backpressure activation threshold
K_q	Final aggregation budget for query q
$K_{\min}$, $K_{\max}$, K_{eff}	Lower / upper / backpressure-adjusted budget bounds
$L_i(K_i)$	Prompt length (tokens) given budget K_i
μ_{p}, μ_{d}	Prefill / decode throughput (tokens / ms)
$p_i = L_i(K_i)/\mu_{\text{p}}$	Estimated prefill time for query q_i
$u(t)$	Instantaneous queue utilization $\in [0,1]$ (Eq. 6)
	Scheduling (Phase 3)
$U_{\text{slo}}(q)$	SLO urgency $= \min(p_q/\tau_q,\ 1)$ (Eq. 13)
$\varphi(\cdot)$	Difficulty-aware amplification function
$U^*_{\text{slo}}(q)$	Amplified urgency $= U_{\text{slo}} \cdot \varphi(S_{\text{diff}})$ (Eq. 14)
$C_{\text{prefill}}(q)$	Normalized prefill cost $= L_q/L_{\text{ref}}$ (Eq. 15)
$\mathcal{P}(q)$	Composite priority score (Eq. 16)
t_{arr}, $t_{\text{wait}}(q)$	Arrival time and elapsed wait time

A NOTATION TABLE

Table 6 is the full notation table for this paper.

B PROOFS OF THEORETICAL RESULTS

This appendix provides complete proofs for the theoretical results stated in the main text.

B.1 Proof of Theorem 1 (NP-Hardness)

Proof. We reduce from the NP-complete Partition problem [29].

Source problem (Partition). Given a multiset $A = \{a_1, \ldots, a_m\}$ of positive integers with $\sum_{j=1}^m a_j = 2B$, decide whether there exists a subset $A' \subseteq A$ such that $\sum_{j \in A'} a_j = B$.

Reduction. Given a Partition instance A, construct a QA-Goodput scheduling instance with query set $\mathcal{S} = \{q_1, \ldots, q_m\}$ and fixed budget vector $\mathbf{K}$:

- Each K_i is chosen such that $L_i(K_i)/\mu = a_i$ (the prefill time of q_i equals a_i);
- $\tau_{q_i} = B$ for all i (common deadline);
- $Q(K_i, q_i) = a_i$ (quality proportional to processing time);
- $\eta = B$ (threshold).

Correctness. ($\Rightarrow$) Suppose a partition A' exists with $\sum_{j \in A'} a_j = B$. Schedule the corresponding queries first in any order. Their total processing time is $\sum_{i:q_i \in A'} L_i(K_i)/\mu = B \leq \tau_{q_i}$, so all complete by the deadline. The QA-Goodput is $\sum_{i \in A'} Q(K_i, q_i) = B \geq \eta$.

($\Leftarrow$) Suppose a serving order π achieves QA-Goodput $\geq \eta = B$. Let S be the set of on-time queries. Then $\sum_{i \in S} Q(K_i, q_i) = \sum_{i \in S} a_i \geq B$. Since all on-time queries complete by deadline B, $\sum_{i \in S} L_i(K_i)/\mu = \sum_{i \in S} a_i \leq B$. Combining: $\sum_{i \in S} a_i = B$, so S corresponds to a valid partition.

Since Partition is NP-complete, the restricted decision problem is NP-hard. Since Eq. 3 additionally optimizes over $\mathbf{K}$, it is at least as hard. □

B.2 Proof of Theorem 2 (Joint Dominance)

Proof. Let $\mathcal{J}(\mathbf{K}, \pi) = \sum_i Q(K_i, q_i) \cdot \mathbb{I}(\text{TTFT}_i(\mathbf{K}, \pi) \leq \tau_{q_i})$ be the QA-Goodput under budget vector $\mathbf{K}$ and non-preemptive schedule π. Let $(\mathbf{K}^*, \pi^*)$ denote an optimal solution to Eq. (3).

Part (i): Scheduling dominance. By optimality of π^* for fixed $\mathbf{K}^*$,

$$\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^*, \pi^{\text{FCFS}}).$$

Part (ii): Budget dominance. $(\mathbf{K}^{\text{fix}}, \pi^*)$ is a feasible candidate in the search space of Eq. (3), since $\mathbf{K}^{\text{fix}} \in [K_{\min}, K_{\max}]^n$ and π^* is a valid non-preemptive schedule. Because $(\mathbf{K}^*, \pi^*)$ is globally optimal, its objective value cannot be smaller:

$$\mathcal{J}(\mathbf{K}^*, \pi^*) \geq \mathcal{J}(\mathbf{K}^{\text{fix}}, \pi^*).$$

Combining parts (i) and (ii) yields the result. □

Note. Theorem 2 is a *problem-level* result: it shows that the optimal joint solution dominates any single-lever baseline within the feasible region of Eq. (3). DynaGraph approximates this optimum via greedy priority scheduling and online budget allocation; the effectiveness of this approximation is validated empirically in Section 5.

B.3 Scheduling Approximation under Simplified Model

Theorem 3: *[Scheduling Approximation] Consider a set of queries $\{q_i\}$ with processing times $\{p_i\}$, values $\{Q(K_i, q_i)\}$, and a common deadline τ on a non-preemptive single machine. Greedy scheduling by value-density achieves $\mathcal{J}_{\text{greedy}} \geq \text{OPT}/2$.* □

Proof. *Step 1 — Knapsack reduction.* With common deadline τ, a subset S of queries is feasibly schedulable if and only if $\sum_{i \in S} p_i \leq \tau$ (non-preemptive, single machine, any ordering within S works as long as the total fits). This is exactly the 0-1 knapsack problem:

$$\max \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i p_i x_i \leq \tau, \quad x_i \in \{0,1\}.$$

Step 2 — Fractional relaxation. Relaxing x_i to $[0,1]$, the fractional optimum V_{frac} is obtained by sorting items in decreasing value-density v_i/p_i and greedily filling the knapsack [14]. Since the integer optimum satisfies $\text{OPT} \leq V_{\text{frac}}$, this provides an upper bound.
Step 3 — Greedy gap analysis. Consider the greedy integral packing in the same v_i/p_i order. Let j be the first item that does not fit. The greedy packing value satisfies $V_{\text{pack}} \geq V_{\text{frac}} - v_j$, because the fractional solution assigns j a fractional value of at most v_j. Meanwhile, the maximum single-item value $v_{\max} = \max_i v_i \geq v_j$.
Step 4 — Combine. Taking the better of the greedy packing and the best single item:

$$V_{\text{greedy}} = \max(V_{\text{pack}},\ v_{\max}) \geq \max(\text{OPT} - v_j,\ v_j) \geq \frac{\text{OPT}}{2},$$

where the last inequality holds because for any $a, b \geq 0$ with $a+b \geq \text{OPT}$, $\max(a,b) \geq \text{OPT}/2$. No assumption on the distribution of processing times $\{p_i\}$ is required. □

Mapping to DynaGraph. In DynaGraph, value $v_i = Q(K_i, q_i)$ is the per-query answer quality, weight $p_i = L_i(K_i)/\mu_p$ is the prefill time (Eq. 4), and the deadline τ corresponds to the SLO target. The theorem is stated under a simplified common-deadline, single-machine surrogate; it provides a design rationale for why value-density scheduling (i.e., prioritizing urgent, low-cost queries) is a reasonable heuristic, rather than a direct end-to-end guarantee for the full online problem (Eq. 3).

B.4 Queue Stability under MMPP Arrivals

Theorem 4: *[Queue Stability] Let arrivals follow an MMPP with peak rate $\lambda_{\max}$ and let μ_p denote the prefill throughput. Under Phase 2's backpressure controller (Eq. 8), the queue process $Q(t)$ is positive recurrent whenever*

$$\lambda_{\max} \cdot \frac{K_{\min}}{\mu_p} < 1. \tag{17}$$

Moreover, the expected queue length satisfies $\mathbb{E}[Q] \leq B/(\epsilon\delta)$, where $\epsilon = \mu_p/K_{\min} - \lambda_{\max}$, δ is the scheduling window length, and B bounds second-order arrival/service variance. □

Proof. We model the system as a single-server queue with state-dependent service times and Markov-modulated arrivals, using a slotted-time model with slot duration δ.

System model. Let $Q(t)$ be the number of requests in the queue at the beginning of slot t. Arrivals follow an MMPP: a background Markov chain $\{Z(t)\}$ with finite state space $\mathcal{Z}$ modulates the arrival rate, producing $A(t) \sim \text{Poisson}(\lambda_{Z(t)}\delta)$ arrivals per slot. Define $\lambda_{\max} = \max_{z\in\mathcal{Z}} \lambda_z$.

The service time for query q in slot t depends on the backpressure state. Under Phase 2's controller (Eq. 8), the per-query budget satisfies $K_q \leq K_{\text{eff}}(t)$. Under sustained congestion ($u(t) \gg \theta_{\text{bp}}$), $K_{\text{eff}}(t) \to K_{\min}$, so the worst-case service time is bounded by $K_{\min}/\mu_p$.

Lyapunov function. Define $V(t) = \frac{1}{2}Q(t)^2$. The one-slot conditional drift is:

$$\Delta V(t) \triangleq \mathbb{E}\big[V(t+1) - V(t) \mid Q(t), Z(t)\big]. \tag{18}$$

Using the dynamics $Q(t+1) = \max(Q(t) - D(t), 0) + A(t)$:

$$\Delta V(t) \leq Q(t) \cdot \big(\mathbb{E}[A(t) \mid Z(t)] - \mathbb{E}[D(t) \mid Q(t)]\big) + B, \tag{19}$$

where $B = \frac{1}{2}\big((\lambda_{\max}\delta)^2 + (\mu_p\delta/K_{\min})^2\big)$ bounds the second-order terms.

Service rate under backpressure. When $Q(t)$ is large (more precisely, when $u(t) > \theta_{\text{bp}}$), Phase 2 drives $K_{\text{eff}}(t) \to K_{\min}$, so every query's service time is bounded by $K_{\min}/\mu_p$. The effective service rate per slot is at least $\mu_p\delta/K_{\min}$ queries.

Negative drift. Substituting into Eq. 19:

$$\Delta V(t) \leq Q(t) \cdot \delta\left(\lambda_{\max} - \frac{\mu_p}{K_{\min}}\right) + B. \tag{20}$$

When $\lambda_{\max} \cdot K_{\min}/\mu_p < 1$ (Eq. 17), define:

$$\epsilon \triangleq \frac{\mu_p}{K_{\min}} - \lambda_{\max} > 0.$$

Then $\Delta V(t) \leq -\epsilon\delta \cdot Q(t) + B$, which is negative for $Q(t) > B/(\epsilon\delta)$.

Foster–Lyapunov criterion. By the Foster–Lyapunov theorem [35], the existence of a Lyapunov function with negative drift outside a finite set $\{Q : Q \leq B/(\epsilon\delta)\}$ implies positive recurrence of the joint process $(Q(t), Z(t))$. Since $\{Z(t)\}$ is ergodic by assumption and the drift condition holds uniformly over all states $z \in \mathcal{Z}$ (because we use peak rate $\lambda_{\max}$), $Q(t)$ is positive recurrent, i.e., $\mathbb{E}[Q] < \infty$.

Quantitative queue-length bound. By the quantitative Foster–Lyapunov theorem [36], taking expectations over the stationary distribution:

$$\mathbb{E}[Q] \leq \frac{B}{\epsilon\delta}. \tag{21}$$

This bound is used in the starvation analysis (Appendix B.5). □

Mapping to DynaGraph. Eq. 17 yields a closed-form deployment rule: the system is stable whenever $K_{\min} < \mu_p/\lambda_{\max}$. This condition is verified once at deployment time using the profiled prefill throughput μ_p and the expected peak arrival rate $\lambda_{\max}$. The key mechanism is Phase 2's backpressure (Eq. 8): under sustained congestion, $K_{\text{eff}}(t) \to K_{\min}$ drives all service times to their minimum, ensuring the queue drains faster than it fills. Without backpressure, a fixed budget $K_{\max}$ would require the much stronger condition $\lambda_{\max} \cdot K_{\max}/\mu_p < 1$, which may be violated under bursty workloads.

B.5 Starvation Bound under Aging

Proposition 5 (Bounded Expected Wait). *Under the optional aging mechanism*

$$\mathcal{P}_{\text{aged}}(q) = \mathcal{P}(q) + \frac{\Delta_{\text{age}}}{\tau_q} \cdot \max\big(t_{\text{wait}}(q) - T_{\text{age}},\ 0\big), \tag{22}$$

with $\Delta_{\text{age}} > 0$ and $T_{\text{age}} \geq 0$, every query q with finite τ_q satisfies:

$$\mathbb{E}[t_{\text{wait}}(q)] \ \leq\ T_{\text{age}} + \frac{\tau_q}{\Delta_{\text{age}}} \cdot \big(\mathcal{P}_{\max} - \mathcal{P}(q)\big),$$

where $\mathcal{P}_{\max} = \max_{q'\in Q} \mathcal{P}(q')$ is the highest base priority in the current queue.

Proof. After waiting T_{age}, the aging term increases $\mathcal{P}_{\text{aged}}(q)$ at rate $\Delta_{\text{age}}/\tau_q$ per unit time. The query is guaranteed to be served once $\mathcal{P}_{\text{aged}}(q) \geq \mathcal{P}_{\max}$, which occurs after at most $(\mathcal{P}_{\max} - \mathcal{P}(q)) \cdot \tau_q/\Delta_{\text{age}}$ additional time. Taking expectations and noting that new arrivals can at most reset $\mathcal{P}_{\max}$ to a bounded value (since all priority components are bounded), the result follows. □

Mapping to DynaGraph. In DynaGraph's priority function (Eq. 16), the components are bounded: $U^*_{\text{slo}} \in [0, 1.5]$ and $C_{\text{prefill}} \in [0, 1]$. With weights $w_{\text{slo}}, w_{\text{cost}} \geq 0$ summing to 1, the maximum priority gap is $\mathcal{P}_{\max} - \mathcal{P}_{\min} \leq w_{\text{slo}} \cdot 1.5 + w_{\text{cost}} \cdot 1$. For typical deployment weights $(w_{\text{slo}}, w_{\text{cost}}) = (0.6, 0.4)$, this gives $\mathcal{P}_{\max} - \mathcal{P}_{\min} \leq 1.3$. Setting $\Delta_{\text{age}} = 0.5$ and $T_{\text{age}} = \tau_q$ yields $\mathbb{E}[t_{\text{wait}}] \leq \tau_q + 2.6 \cdot \tau_q = 3.6 \cdot \tau_q$, ensuring that even the lowest-priority query is served within a small multiple of its SLO deadline. In practice, backpressure under sustained congestion (Eq. 8) compresses all budgets toward $K_{\min}$, equalizing processing times and substantially reducing the priority gap before aging is needed.

B.6 Design Rationale for Exponential Backpressure

The following classical result from online knapsack theory motivates DynaGraph's choice of exponential decay for the budget ceiling $K_{\text{eff}}(t)$ (Eq. 8).

Theorem 6:*[Optimal Reservation for Online Knapsack [62]] Consider a stream of items arriving online, each with value-to-weight ratio $v \in [L, U]$, and a knapsack of capacity C.* **(a)** *No deterministic online algorithm achieves a competitive ratio better than* $\ln(U/L) + 1$ [62]. **(b)** *The exponential threshold* $\Psi(z) = L \cdot (U/L)^z$*, which admits an item iff* $v \geq \Psi(z)$ *at utilization* $z \in [0, 1]$*, achieves this bound and is therefore optimal* [10]. **(c)** *A linear threshold achieves only* $O(\sqrt{U/L})$*, which is strictly worse when* $U/L > e$ [51]. □

Proof. **Part (a): Lower bound.** Consider a single knapsack of capacity C and items with value-to-cost ratio in $[L, U]$. An adversary constructs two input sequences: σ_1 consists of items with ratio L filling the knapsack, followed by items with ratio U; σ_2 consists only of items with ratio L. Any deterministic algorithm that reserves capacity for high-value items (performing well on σ_1) loses on σ_2, and vice versa. Balancing across all such adversarial constructions yields the lower bound $\ln(U/L) + 1$ [62].

Part (b): Optimality of the exponential threshold. Define the threshold function $\Psi : [0, 1] \to [L, U]$ as:

$$\Psi(z) = L \cdot \left(\frac{U}{L}\right)^z, \tag{23}$$

where $z \in [0, 1]$ is the fraction of capacity consumed. The algorithm admits item i with ratio v_i if and only if $v_i \geq \Psi(z)$.

We verify the competitive ratio via the primal-dual method of Chakrabarty et al. [10]. Assign a dual variable $\psi(z) = \Psi(z)$ to the capacity constraint. For any admitted item i at utilization z_i, the primal gain is $v_i \geq \Psi(z_i)$. Integrating the dual over the utilization trajectory:

$$\int_0^Z \Psi(z)\, dz = \int_0^Z L \cdot \left(\frac{U}{L}\right)^z dz = \frac{L}{\ln(U/L)} \left[\left(\frac{U}{L}\right)^Z - 1\right],$$

where Z is the final utilization. The ratio of primal value to dual cost is bounded by $\ln(U/L) + 1$ for all input sequences, matching the lower bound from part (a).

Part (c): Sub-optimality of the linear threshold. A linear threshold $\Psi_{\text{lin}}(z) = L + (U - L) \cdot z$ admits items with ratio in $[L, L + (U - L)z]$ at utilization z. Under an adversarial arrival sequence that concentrates high-value items ($v_i \approx U$) at $z \to 1$, the algorithm has already consumed capacity on medium-value items, yielding a competitive gap of $\Theta(\sqrt{U/L})$. Yang et al. [51] provide the tight analysis for multidimensional knapsack, which specializes to the single-dimension case here. □

Mapping to DynaGraph. In DynaGraph, each query q corresponds to an "item" with value $Q(K_q, q)$ (answer quality) and cost proportional to K_q (prefill tokens). The ratio range $[L, U]$ maps to $[K_{\min}, K_{\max}]$. DynaGraph's effective ceiling (Eq. 8) instantiates Ψ in budget space:

$$K_{\text{eff}}(u) = K_{\min} + (K_{\max} - K_{\min}) \cdot \exp\left(-\ln \frac{K_{\max}}{K_{\min}} \cdot \frac{u - \theta_{\text{bp}}}{1 - \theta_{\text{bp}}}\right),$$

which is monotonically decreasing in u and bounded in $[K_{\min}, K_{\max}]$. The shift by θ_{bp} defines a free zone ($u \leq \theta_{\text{bp}}$) where capacity is unconstrained, following the slack extension of Buchbinder and Naor [9]. This result *motivates* the choice of exponential decay over linear alternatives (validated empirically in Section 5.5, Figure 7(a)), rather than serving as a direct end-to-end guarantee for the full online problem.

Compatibility with Queue Stability. The queue stability guarantee (Theorem 4) uses $K_{\min}$ as the worst-case budget under saturation. Since $K_{\text{eff}}(t)$ is monotonically decreasing and $K_{\text{eff}}(t) \to K_{\min}$ as $u(t) \to 1$, the Lyapunov argument applies directly with $K_{\min}$ as the service-time bound.

C DETAILED EXPERIMENT SETTING

This appendix provides the hardware and software stack, workload protocol, dataset and model details, and DynaGraph's configuration defaults used throughout the evaluation. Baseline-specific invocation details and vLLM launch commands are available in the code repository [4].

C.1 Hardware and Software Stack

Table 7: Hardware and software environment.

Component	Specification
GPU	2× NVIDIA RTX 4090 (24 GB each)
Host memory	128 GB DDR5
CUDA	12.8
Python	3.11
PyTorch	2.10.0
LLM engine	vLLM 0.17.1 (main) / SGLang ≥0.5.9
Embedding index	FAISS (CPU) 1.13.2

All tensor-parallel-2 runs pin both GPUs. We use the FLASHINFER attention backend for Qwen3.5 and Qwen3 models with FP8 KV-cache (–kv-cache-dtype fp8). Priority scheduling is enabled via –scheduling-policy priority (vLLM) or –enable-priority-scheduling (SGLang).

C.2 Workload Protocol

Unless otherwise stated for a specific RQ, all experiments use the following online serving protocol.

Table 8: Default workload configuration.

Parameter	Value
Arrival process	Zipf popularity, $\alpha = 1.0$
Unique queries $\lvert\mathcal{S}\rvert$	1 000
Total requests N	3 000
Request rate λ	30 qps
Client concurrency	64
TTFT deadlines τ	{200, 500, 2 000} ms
Warm-up (discarded)	300 requests
DynaGraph $K_{\min}$ / $K_{\max}$	10 / 20
DynaGraph θ_{bp}	0.7
DynaGraph decay shape	Exponential (Eq. 8)

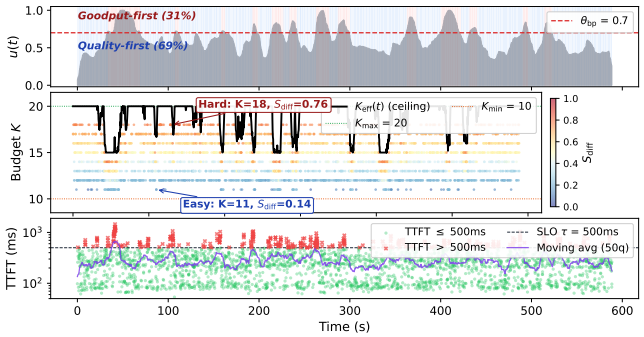


Figure 8: Closed-loop budget allocation in action. Top: queue utilization $u(t)$ alternates between quality-first and goodput-first regimes. Middle: $K_{\text{eff}}(t)$ decays under congestion while per-query K_q preserves difficulty differentiation (darker = harder). Bottom: resulting per-query TTFT.

Each run consists of a 300-request warm-up (discarded) followed by the 3 000-request measured segment. Metrics are computed on the measured segment only. Per-RQ deviations: RQ4 varies $K_{\min} \in [3, 15]$, $\theta_{\text{bp}} \in [0.1, 1.0]$, and $\lambda \in [2, 30]$; RQ5 varies the LLM backbone and operating λ.

C.3 Datasets

Table 9: Dataset statistics.

Dataset	Full queries	Full corpus	Sampled	Used in
2WikiMultihopQA	12 576	56 687	6 119	RQ1–RQ7
HotpotQA	7 405	66 635	9 811	RQ1, RQ3–RQ7
MuSiQue	2 417	11 656	11 656	RQ1, RQ5

Following the HippoRAG protocol [19], we randomly sample 1 000 queries per dataset with their supporting and distractor passages as the retrieval corpus $\mathcal{D}$ (≈6–12 K passages per dataset). For RQ5 corpus scalability on 2Wiki, we additionally use the full development set (12 576 queries; ≈57 K passages).

C.4 Models

All model weights are downloaded from the Hugging Face Hub.

Table 10: Models used in evaluation.

Role	Model	Used in
Default LLM (hybrid Mamba)	Qwen3.5-4B	RQ1, RQ3, RQ5, RQ6
Pure-attention LLM	Qwen3-4B-Instruct-2507	RQ2, RQ4, RQ7
Medium LLM	Qwen2.5-7B-Instruct	RQ2
Cross-scale LLMs	Qwen3.5-{0.8B, 2B, 9B, 35B-A3B}	RQ5
Embedding	Qwen3-Embedding-0.6B	Phase 1 retrieval
Reranker	BGE-Reranker-v2-M3	Phase 1 rerank

C.5 DynaGraph Configuration

Tables 11 and 12 list the default configuration for DynaGraph's Phase 2 controller and Phase 3 scheduler. All ablation experiments (RQ3, RQ4, RQ7) vary one parameter at a time while keeping the rest at these defaults.

Table 11: Phase 2 controller defaults.

Parameter	Description	Default
$K_{\min}$, $K_{\max}$	Budget bounds	10, 20
θ_{bp}	Backpressure threshold	0.7
Decay shape	$K_{\text{eff}}(t)$ under congestion	Exponential
K mapping	$S_{\text{diff}} \to K_q$	Linear
(α, β, γ)	Weights of $(r_{\text{bridge}}, \rho_{\text{overlap}}, n_{\text{entity}})$	(0.4, 0.3, 0.3)

Table 12: Phase 3 scheduler defaults.

Parameter	Description	Default
Strategy	Priority computation	Composite
w_{slo}	Weight on amplified urgency U^*_{slo}	0.6
w_{cost}	Weight on prefill cost C_{prefill}	0.4
Difficulty amplification	Boost U_{slo} when $S_{\text{diff}} > 0.5$	Enabled
μ_{p}	Calibrated prefill throughput	Per-model profiled

C.6 Per-RQ Experiment Matrix

Table 13 provides a compact overview of each RQ's LLM, tensor parallelism, datasets, and comparison targets.

Table 13: Per-RQ experiment configuration summary.

RQ	LLM	TP	Datasets	Compared against
RQ1	Qwen3.5-4B	2	2Wiki, HotpotQA, MuSiQue	SiReRAG, KET-RAG, ClueRAG, HippoRAG
RQ2	Qwen3-4B, Qwen2.5-7B	2	2Wiki	CacheBlend, DepCache, VanillaGRAG
RQ3	Qwen3.5-4B	2	2Wiki	Decay / feature ablations
RQ4	Qwen3-4B	1	2Wiki	Static baseline (K=20+FCFS)
RQ5	Qwen3.5-{0.8–35B}	2	2Wiki (+full corpus)	Per-model capacity sweep
RQ6†	Qwen3.5-4B	2	2Wiki, HotpotQA	Adaptive-K, RAG-CSM, CompSelect
RQ7†	Qwen3-4B	1	2Wiki	FCFS, SJF, EDF, Phase ablations

†Appendix experiments (Section D).

D ADDITIONAL EXPERIMENT RESULTS

This appendix provides two supplementary experiments that extend the main evaluation (Section 5). RQ6 (Section D.1) compares DynaGraph's context selection against training-free and trained baselines under the same pre-computed retrieval setting as RQ1–RQ4, isolating the contribution of Phase 2's difficulty-aware budgeting from Phase 3's scheduling. RQ7 (Section D.2) provides detailed phase ablations (Phase 2 vs Phase 3) and scheduling strategy comparisons (FCFS, SJF, EDF, composite) on two different LLM architectures, supplying the fine-grained evidence behind the joint design claims in Section 3.2 and Theorem 2.

D.1 RQ6: Context Aggregation Comparison

RQ6 compares DynaGraph's context selection against four baselines spanning three design families: static budgets, training-free score thresholding, and trained selectors. All methods use the same Phase 1 retrieval output and FCFS scheduling (no Phase 3) unless otherwise noted, isolating the effect of context selection. Experiments are conducted on 2Wiki and HotpotQA with Qwen3.5-4B at Zipf@30 QPS (Table 14).

DynaGraph vs training-free baselines. On 2Wiki, DynaGraph achieves $\mathcal{J}$@500 of 81.8%, outperforming the strongest training-free baseline (Uniform K=10: 57.5%) by +24.3 pp, while using a comparable average budget (avg K: 10.1 vs 10.0) and maintaining similar F1 (0.781 vs 0.780). Adaptive − K [43] thresholds on reranker score ratios, producing avg K=11.2, yet achieves only

Table 15: Phase ablation (Qwen3.5-4B, 2Wiki, Zipf@30 QPS). Configurations follow Table 1; C4 automates expert labels via Phase 1 topological features with backpressure feedback.

Config	K	Sched	F1	Acc	p50 (ms)	SLO@500	$\mathcal{J}$@500
C1 (Static)	Fixed 20	FCFS	**.800**	**.857**	5892	0.0%	0.0%
C2 (P3 only)	Fixed 20	Composite	.793	.859	407	57.8%	41.6%
C3 (P2 only)	Adaptive	FCFS	.787	.844	4076	0.0%	0.0%
C4 (DynaGraph)	Joint	+BP	.787	.841	**220**	**94.9%**	**72.8%**

Table 14: Context aggregation comparison (Qwen3.5-4B TP2, Zipf@30 QPS, 1000q×3000). Baselines use FCFS scheduling; DynaGraph uses joint Phase 2+3.

Method	F1↑	avg K	p50 (ms)↓	p99 (ms)↓	SLO@500↑	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑	$\mathcal{J}$@2s↑
2WikiMultiHopQA								
Full K=20 (C1)	.784	20.0	305	9127	83.5%	24.7%	52.5%	61.5%
Uniform K=10	.780	10.0	223	1740	94.1%	26.3%	57.5%	60.6%
Adaptive-K [43]	.788	11.2	229	3220	94.1%	24.2%	57.0%	59.6%
RAG-CSM [15]	.522	6.6	162	2385	97.7%	28.7%	46.8%	47.3%
CompSelect [58]	.328	2.7	107	1155	98.2%	28.1%	31.0%	31.4%
DynaGraph	.781	10.1	215	15097	98.6%	**41.5%**	**81.8%**	**82.8%**
HotpotQA								
Full K=20 (C1)	.776	20.0	220	2032	95.6%	28.6%	68.1%	70.7%
Uniform K=10	.767	10.0	220	2034	94.8%	28.5%	66.8%	69.9%
Adaptive-K [43]	.767	10.0	220	2036	96.0%	27.7%	67.4%	69.5%
RAG-CSM [15]	.555	6.4	158	2005	97.8%	35.1%	47.2%	47.8%
CompSelect [58]	.602	5.0	115	5090	97.9%	46.2%	51.1%	51.1%
DynaGraph	.765	10.1	220	2782	95.9%	29.9%	**68.3%**	**70.3%**

57.0% $\mathcal{J}$@500—on par with Uniform K=10. This confirms that context selection quality alone (even with near-perfect offline NDCG of 0.974) does not translate to online goodput without serving-aware scheduling. On HotpotQA, all training-free methods cluster within 66.8–68.3% $\mathcal{J}$@500, with DynaGraph marginally ahead (68.3%); this narrower gap reflects HotpotQA's lower multi-hop complexity, where most queries are answerable with moderate context and scheduling provides less leverage.

Trained context selectors. RAG – CSM [15] trains a BERT-based scorer on per-document context influence, compressing avg K to 6.6 (−67% vs K=20) and achieving the second-lowest p50 (162 ms). However, F1 drops to 0.522 (−26 pp vs DynaGraph), indicating that the learned scorer over-prunes bridging evidence required for multi-hop reasoning. CompSelect [58] applies LoRA-tuned sentence extraction, reducing avg K to 2.7 (−86%) with the lowest p50 (107 ms), but F1 collapses to 0.328 (−45 pp), yielding $\mathcal{J}$@500 of only 31.0%. These results reveal a *K-cliff* phenomenon: reducing avg K from 10 to 6 incurs a 26 pp F1 penalty, and from 10 to 3 a 45 pp penalty, whereas reducing from 20 to 10 costs only 0.4 pp. Gold answer documents are retained in the top-K, but the bridging context required for multi-hop chains falls outside the aggressively compressed budget.

Retrieval-serving gap. Adaptive – K achieves the highest offline retrieval quality (NDCG@10: 0.974 on 2Wiki), yet delivers identical $\mathcal{J}$@500 as Uniform K=10 (57.0% vs 57.5%). DynaGraph's NDCG@10 is lower (0.941), but its $\mathcal{J}$@500 is 81.8%—a +24.8 pp gap. This *retrieval-serving gap* demonstrates that offline ranking metrics cannot predict online goodput; the critical differentiator is queue-aware scheduling (Phase 3), which converts the per-query cost variance created by difficulty-aware K allocation (Phase 2) into system-level goodput gains.

Complementarity with trained methods. Both RAG – CSM and CompSelect are orthogonal to DynaGraph's training-free pipeline: their learned scoring functions could serve as drop-in replacements for the reranker in Phase 1, potentially improving the quality of the candidate set C_q that Phase 2 subsequently compresses. However, their current implementations assume static K and FCFS scheduling, and integrating them with load-aware backpressure and priority scheduling remains unexplored. The results suggest that the largest gains lie not in better context selection per se, but in coupling any selection method with serving-aware execution.

D.2 RQ7: Phase Ablation and Scheduling Strategy Analysis

The main text (RQ2) establishes that DynaGraph's closed-loop interaction between Phase 2 (adaptive budgeting) and Phase 3 (SLO-aware scheduling) is necessary for high QA-Goodput. RQ7 provides detailed ablations supporting this claim: (A) a phase-level ablation isolating Phase 2 and Phase 3 on Qwen3.5-4B (Table 15), and (B) a scheduling strategy comparison decomposing each component's contribution on Qwen3-4B (Table 16).

(A) Phase ablation (Table 15). We ablate Phase 2 and Phase 3 on Qwen3.5-4B (hybrid Mamba, TP=2) using four configurations that follow the progressive elimination structure in Table 1: C1 (Fixed K=20 + FCFS), C2 (Fixed K=20 + composite scheduling), C3 (adaptive K + FCFS), and C4 (DynaGraph: adaptive K + composite scheduling + backpressure). Neither KV-cache baseline (CacheBlend, DepCache) can run on Qwen3.5-4B's Mamba layers, making this ablation the only way to evaluate serving strategies on hybrid architectures.

(A1) Phase 2 alone reduces cost but cannot meet SLOs. C3 applies difficulty-aware K allocation under FCFS, reducing avg K from 20 to 14.7 and lowering mean prefill cost by 27%. However, SLO@500 remains 0% (p50: 4,076 ms) because FCFS allows a small number of heavy queries—those with both high prefill and long decode—to block the queue. Adaptive budgeting creates per-query cost heterogeneity, but without a scheduler to exploit it, head-of-line blocking persists.

(A2) Phase 3 alone improves SLO but wastes budget. C2 applies composite priority scheduling with uniform K=20, lifting SLO@500 to 57.8% and p50 from 5,892 ms to 407 ms. However, with every query allocated the same budget, there is no cost variance to exploit: the scheduler can reorder but cannot reduce aggregate prefill load, capping $\mathcal{J}$@500 at 41.6%. This confirms the insight from Section 3.2: scheduling uniformly heavy requests merely redistributes latency without reducing it.

(A3) Joint Phase 2+3 resolves both bottlenecks. C4 (DynaGraph) combines adaptive K with composite scheduling and backpressure. Adaptive K compresses easy queries while preserving larger budgets for structurally complex ones, creating per-query cost variance; the scheduler interleaves short and long prefills to minimize queue waiting time; under congestion, backpressure further compresses K_{eff} (Eq. 8). The result: SLO@500 reaches 94.9% and $\mathcal{J}$@500 reaches 72.8%, with F1 only −0.013 below C1. This validates Theorem 2: the joint solution strictly dominates either phase in isolation.

(A4) Architecture generality. Because the optimization operates above the inference engine—controlling prompt length and request ordering rather than modifying attention computation—DynaGraph extends to hybrid architectures such as Qwen3.5-4B's Mamba layers. KV-cache methods (CacheBlend, DepCache) de-

Table 17: End-to-end live deployment. Retrieval and generation run concurrently. Qwen3.5-4B and Qwen3-4B, λ=5 qps, 1 000 queries.

LLM	Dataset	Retr (ms)	Gen (ms)	E2E p50	E2E p95	F1	Acc
Qwen3.5-4B	2Wiki	178	414	613	1 157	.797	.842
	HotpotQA	270	231	529	724	.714	.670
	MuSiQue	300	339	679	922	.399	.421
Qwen3-4B	2Wiki	180	304	502	888	.764	.803
	HotpotQA	254	138	395	525	.698	.650
	MuSiQue	352	276	614	836	.371	.341

Table 16: Scheduling strategy comparison (2Wiki, Qwen3-4B TP1, Zipf@30 QPS, 1000q×3000 requests). Each method uses its natural K strategy.

Method	K strat.	Sched	F1↑	avg K	p50↓	p95↓	p99↓	$\mathcal{J}$@200↑	$\mathcal{J}$@500↑
C1 (Static)	Fixed 20	FCFS	.759	20.0	359	1560	4122	19.0%	54.3%
SJF K=20	Fixed 20	SJF	.756	20.0	344	2022	4228	21.3%	53.7%
EDF K=20	Fixed 20	EDF	.756	20.0	329	1034	5469	20.8%	60.3%
SJF K=10	Fixed 10	SJF	.742	10.0	133	343	1465	61.5%	78.0%
C3 (AdpK only)	Diff, no BP	FCFS	.750	14.7	181	501	3000	45.5%	76.5%
AdpK+BP	Diff+BP	FCFS	.755	12.5	173	476	2969	47.8%	77.4%
DynaGraph	Diff+BP	Comp.	.755	12.5	**143**	**427**	3310	**55.6%**	**77.9%**

pend on specific attention layouts and cannot run on this architecture, as shown in RQ2.

(B) Scheduling strategy comparison (Table 16).

We compare scheduling strategies on Qwen3-4B (TP1, higher per-query load pressure) to quantify each component's marginal contribution. Each method uses its natural K allocation to reflect realistic deployment. The table decomposes the path from C1 (Static) to DynaGraph into four incremental steps.

(B1) Scheduling alone has limited effect with uniform K. With fixed K=20, replacing FCFS with SJF improves $\mathcal{J}$@200 by only +2.3 pp (21.3% vs 19.0%), and EDF by +1.8 pp. When all prompts are uniformly heavy, the scheduler can reorder requests but cannot reduce total prefill load. SJF slightly improves median latency (344 vs 359 ms) but worsens p95 (2,022 vs 1,560 ms), because prioritizing short jobs starves long ones. EDF improves p95 (1,034 ms) by respecting deadlines but at the cost of p99 (5,469 ms), as late-arriving hard queries miss their window. Neither strategy achieves meaningful $\mathcal{J}$@500 improvement (53.7–60.3% vs C1's 54.3%). This validates Obs. 3 (Section 3.2): scheduling uniformly heavy requests merely redistributes latency across the queue.

(B2) Adaptive K is the dominant lever. C3 (difficulty-aware K, no backpressure, FCFS) lifts $\mathcal{J}$@200 from 19.0% to 45.5% (+26.5 pp) and $\mathcal{J}$@500 from 54.3% to 76.5% (+22.2 pp), dwarfing any scheduling-only improvement. The gain comes from two sources: easy queries receive smaller budgets and complete faster, reducing their own latency; simultaneously, hard queries receive proportionally larger budgets, preserving the evidence needed for multi-hop reasoning (F1: 0.750 vs C1's 0.759, only −0.9 pp). Graph-topological difficulty scoring (Phase 2) is the primary contributor to online goodput.

(B3) Backpressure adds load-proportional headroom. Adding the Phase 3→Phase 2 backpressure loop (C3→AdpK+BP) improves $\mathcal{J}$@200 by +2.3 pp (47.8% vs 45.5%) and reduces avg K from 14.7 to 12.5 (−15%) by dynamically compressing K_{eff} during congestion bursts. The improvement is modest at this load level (λ=30), consistent with RQ4's finding that backpressure contributes a load-proportional complement: its impact grows with request rate. Crucially, F1 *increases* from 0.750 to 0.755 despite lower avg K, because the difficulty-aware allocation under backpressure redistributes budget from over-provisioned easy queries to under-served hard ones.

(B4) Composite priority improves median and tail latency. DynaGraph's composite priority adds +7.8 pp $\mathcal{J}$@200 over FCFS with the same adaptive K (55.6% vs 47.8%), while reducing p50 from 173 to 143 ms (−17%) and p95 from 476 to 427 ms (−10%). The composite score combines estimated cost, deadline slack, and queue age, allowing the scheduler to interleave short easy prefills between long hard ones rather than processing them in arrival order.

(B5) Brute-force compression is suboptimal. SJF with K=10 achieves the highest $\mathcal{J}$@200 (61.5%) among all configurations by aggressively shortening all prompts, surpassing DynaGraph's 55.6%. However, this comes at the cost of the lowest F1 in the table (0.742 vs DynaGraph's 0.755, −1.3 pp), because all queries—including hard multi-hop ones—receive the same compressed budget. Under relaxed SLO (τ=500 ms), DynaGraph closes the gap (77.9% vs 78.0%, within 0.1 pp), demonstrating that *selective* budget allocation preserves quality while approaching the latency of brute-force compression. This underscores the core design principle: context compression alone is insufficient; *difficulty-aware* compression coupled with serving-aware scheduling maximizes the quality–latency frontier.

Summary. The two ablations, conducted on different LLM architectures, converge on the same conclusion: (1) adaptive K is the dominant factor, providing the cost variance that all downstream optimizations depend on; (2) backpressure and composite scheduling each add incremental gains that compound under load; (3) the joint system strictly dominates any single-lever approach across both strict (τ=200 ms) and moderate (τ=500 ms) SLO regimes. These results provide the detailed evidence for the design claims in Section 3.2 and the closed-loop architecture in Section 4.3–4.4.

D.3 End-to-End Live Pipeline Deployment

RQ1–RQ4 and RQ6–RQ7 use pre-computed retrieval to isolate serving-layer effects. Table 17 complements RQ5 by reporting fully live end-to-end latency, where graph construction, embedding computation, reranking, and LLM inference run concurrently on the same hardware.

E2E p50 stays below 700 ms on every dataset–LLM combination. Retrieval contributes 178–352 ms depending on subgraph density (MuSiQue > HotpotQA > 2Wiki), while generation time scales with prompt length. F1 matches the pre-computed-retrieval numbers from Table 3 within 0.02, confirming that live retrieval does not erode answer quality. The pure-attention Qwen3-4B achieves lower E2E latency than the hybrid-Mamba Qwen3.5-4B on HotpotQA (395 vs 529 ms) because its shorter retrieved contexts yield faster prefill under the attention-only architecture.

D.4 Cross-System Scalability Comparison

Figure 7(d) in the main text shows that DynaGraph's E2E latency grows sub-linearly under corpus expansion. Table 18 quantifies this advantage against ClueRAG, the strongest retrieval-optimized baseline with released scalability data, on identical HotpotQA corpus

Table 18: E2E scalability: DynaGraph vs ClueRAG on HotpotQA with increasing corpus size. Ratio = ClueRAG / DynaGraph E2E p50.

Scale	Queries	Corpus	DynaGraph p50 (ms)	ClueRAG p50 (ms)	Ratio
1×	1 000	~9.8 K	360	609	1.7×
3×	3 000	~28 K	369	963	2.6×
6×	6 000	~55 K	386	1 614	4.2×
Full	7 405	~67 K	392	2 076	**5.3**×

subsets (Qwen3.5-4B, TP=1, λ=3 qps).

DynaGraph's E2E p50 increases by only 32 ms (360→392 ms, +9%) as the corpus grows 7×, because FAISS-based two-hop expansion with a fixed reranker window bounds retrieval cost. In contrast, ClueRAG's iterative traversal grows close to linearly (609→2 076 ms, +241%), widening the gap from 1.7× at 1× to 5.3× at full corpus. This confirms the sub-linear scaling claim in Section 5.6 and demonstrates that DynaGraph's retrieval architecture is better suited for production-scale corpora where corpus size grows continuously.